
Licenciatura em Engenharia Biomédica
Ano letivo 2025 / 2026

LabSystem

Sistema de Gestão de Laboratório
de Análises Clínicas
Relatório de Projeto

Unidade Curricular: INFMED
Docente:

Gonçalo Pires

Nº 1220673 · Turma C

Manuel Antunes

Nº 1220769 · Turma C

Instituto Superior de Engenharia do Porto · Junho de 2026

Índice

1. Introdução	5
1.1. Contexto e motivação	5
1.2. Objetivos do projeto.....	5
2. Estado da Arte	5
2.1. Informática médica e sistemas de informação em saúde	5
2.2. O registo clínico eletrónico (EMR)	6
2.3. Saúde digital e o portal do utente.....	6
2.4. Tecnologias adotadas	6
3. Requisitos do Sistema	7
3.1. Requisitos Funcionais	7
3.2. Requisitos Não Funcionais	8
3.3. Casos de Uso	9
4. Arquitetura do Sistema	9
4.1. Visão geral	9
4.2. Diagrama de componentes	9
4.3. Vista Lógica	10
4.4. Vista de Implantação	11
4.5. Integração e validação de dados em XML/XSD	12
5. Modelo de Domínio.....	13
5.1. User (utilizador).....	14
5.2. Utente (paciente).....	15
<i>Tabela 4 - Entidade Utente.</i>	15
5.3. Requisição.....	15
5.4. Amostra.....	16
5.5. Resultado.....	16
5.6. Fatura	17
<i>Tabela 8 - Entidade Fatura.</i>	17
5.7. Equipamento, Reagente e Manutencao	17
5.8. AuditLog e Notification	17
6. Fluxos de Trabalho Clínicos.....	19
6.1. Fluxo completo.....	19
6.2. Diagrama de estados da Amostra	19
6.3. Diagrama de estados do Resultado	19
6.4. Diagrama de estados da Fatura.....	20
6.5. Atribuição Médico–Utente.....	20
7. Segurança e Controlo de Acessos.....	20
7.1. Modelo RBAC.....	20
7.2. Autenticação JWT e versionamento de token	21
7.3. Proteção das palavras-passe e do login	21
7.4. Histórico de ações no sistema.....	21
8. Interface Gráfica - Demonstração das Funcionalidades	21

8.1. Ecrã inicial do Administrador.....	21
8.2. Página de Login.....	22
8.3. Recuperação de Password.....	22
8.4. Registo / Criação de Conta.....	23
8.5. Portal do Utente.....	23
8.6. Portal Médico	24
8.7. Portal Técnico	24
8.8. Módulo Utentes	25
8.9. Módulo Requisições	25
8.10. Módulo Colheita.....	26
8.11. Módulo Análise.....	27
8.12. Módulo Validação	28
8.13. Módulo Financeiro	28
8.14. Módulo Equipamentos.....	28
8.15. Módulo Analytics.....	29
8.16. Módulo Utilizadores	29
8.17. Módulo Segurança.....	30
8.18. Módulo Integrações	30
8.19. Notificações em Tempo Real	31
8.20. Intercâmbio de dados em XML.....	31
9. Testes	32
9.1. Estratégia e ferramenta de teste.....	32
9.2. Casos de teste implementados	32
10. Conclusão.....	33
10.1. Síntese e objetivos atingidos	33
10.2. Limitações e trabalho futuro	33
Referências.....	33
Anexo A - Esquemas XSD.....	35
Anexo B - Exemplos de XML.....	37

Índice de Figuras

Figura 1 – Diagrama de casos de uso do LabSystem.....	9
Figura 2 – Diagrama de componentes e interfaces de comunicação.....	10
Figura 3 – Vista lógica — arquitetura em camadas do software.....	11
Figura 4 – Vista de implantação — três componentes ligados por API REST.....	12
Figura 5 – Diagrama de classes do domínio do LabSystem.....	14
Figura 6 – Fluxo clínico completo e máquina de estados	19
Figura 7 – Diagrama de estados da Amostra.....	19
Figura 8 – Diagrama de estados do Resultado.....	20
Figura 9 – Diagrama de estados da Fatura	20
Figura 10 – Ecrã inicial do administrador após autenticação	22
Figura 11 – Página de login.....	22
Figura 12 – Recuperação de palavra-passe	23
Figura 13 – Criação de conta (pendente de aprovação)	23
Figura 14 – Portal do utente	24
Figura 15 – Portal médico	24
Figura 16 – Portal técnico	25
Figura 17 – Módulo de utentes.....	25
Figura 18 – Módulo de requisições.....	26
Figura 19 – Módulo de colheita.....	26
Figura 20 – Portal de enfermagem — registo de colheita	27
Figura 21 – Módulo de análise — worklist.....	27
Figura 22 – Módulo de validação.	28
Figura 23 – Módulo financeiro.....	28
Figura 24 – Módulo de equipamentos e reagentes.....	30
Figura 25 – Módulo de analytics	30
Figura 26 – Módulo de gestão de utilizadores.....	31
Figura 27 – Módulo de segurança — histórico de ações.....	31
Figura 28 – Módulo de integrações	32
Figura 29 – Painel de notificações, com contador de não lidas.....	32
Figura 30 – Importação de XML com validação contra o esquema XSD (com ramo de erro 422).....	33

1. Introdução

1.1. Contexto e motivação

Os laboratórios de análises clínicas constituem um pilar do diagnóstico médico moderno, estimando-se que uma parte muito significativa das decisões clínicas seja suportada por resultados laboratoriais. O crescente volume de pedidos, a exigência de rastreabilidade total da amostra do utente ao resultado e a necessidade de integração com os sistemas do Serviço Nacional de Saúde (SNS) e seguros tornaram indispensável a informatização rigorosa destes processos através de Sistemas de Informação Laboratorial (LIS - *Laboratory Information Systems*).

O LabSystem foi concebido para apoiar a operação integral de um laboratório de análises clínicas, materializando, de forma funcional, um LIS completo e moderno. O projeto aplica arquitetura em camadas, API REST, autenticação segura, autenticação segura e separação em camadas a um domínio real e exigente como o da saúde, no qual coexistem múltiplos intervenientes, estados de negócio bem definidos, eventos sensíveis ao tempo e requisitos legais estritos de privacidade.

1.2. Objetivos do projeto

Definiram-se os seguintes objetivos:

- Implementar, de ponta a ponta, o fluxo clínico requisição → colheita → análise → validação → relatório → faturação;
- Garantir segurança e privacidade através de autenticação JWT, controlo de acessos baseado em papéis (RBAC) e registo de auditoria, em conformidade com o RGPD;
- Disponibilizar uma API REST estruturada e testável, preparada para a troca de dados com sistemas externos;
- Notificar os intervenientes perante eventos relevantes, em especial resultados com valor crítico;
- Assegurar a separação clara entre frontend, backend e base de dados, comunicando exclusivamente por API REST.

2. Estado da Arte

2.1. Informática médica e sistemas de informação em saúde

A informática médica engloba o conjunto de saberes das áreas da engenharia e da informática aplicados aos problemas da saúde, organizando-se, à semelhança do ciclo clínico, em três etapas: entrada de dados, processamento e saída de resultados. Neste âmbito, um sistema de informação em saúde é um sistema automatizado ou manual que reúne pessoas, máquinas e métodos para recolher, processar, transmitir e disseminar dados que representam informação útil para o utilizador [1].

Tal como os recursos humanos, materiais e financeiros, a informação é hoje um recurso que tem de ser gerido com eficiência pelas organizações de saúde, sendo a prestação de cuidados, cada vez mais, um serviço dirigido pela informação [1]. Esta relação entre dados e decisão é habitualmente ilustrada pela pirâmide DIKW: os dados são registos brutos e isolados; a

informação resulta da sua organização e contextualização; o conhecimento surge da interpretação da informação, revelando padrões e relações; e a sabedoria corresponde à aplicação criteriosa do conhecimento na tomada de decisão. Num laboratório de análises clínicas, é precisamente este percurso de um valor medido até uma decisão clínica fundamentada que um sistema de informação deve suportar de forma fiável, rastreável e segura.

Foi esta relevância que motivou a escolha do tema. Por um lado, trata-se de um domínio de impacto real, em que a correta gestão da informação se traduz diretamente na segurança do doente e na qualidade dos cuidados. Por outro, é um domínio tecnicamente rico, que reúne num só sistema vários perfis de utilizador com permissões distintas, processos com estados bem definidos, eventos sensíveis ao tempo e exigências de privacidade designadamente o cumprimento do Regulamento Geral sobre a Proteção de Dados [10] o que faz do seu desenvolvimento um exercício particularmente completo.

2.2. O registo clínico eletrónico (EMR)

O registo clínico eletrónico (*Electronic Medical Record*) é um registo que contém a informação clínica de saúde e de doença de um paciente, com notas elaboradas por médicos, enfermeiros e outros profissionais, resultados de meios complementares de diagnóstico e informação sobre o tratamento [2]. Entre os seus componentes contam-se a identificação do paciente, o histórico médico, as prescrições, as notas clínicas, o agendamento e os resultados de exames e testes, nomeadamente os exames laboratoriais.

É exatamente este último componente que o LabSystem produz e gere: ao tratar todo o ciclo da análise clínica, da requisição ao resultado validado, o sistema preenche o registo clínico do utente com informação laboratorial estruturada e validada. As funcionalidades chave de um sistema desta natureza identificação inequívoca de amostras, organização do trabalho por equipamento, validação dos resultados com base em intervalos de referência e sinalização de valores críticos refletem-se diretamente no desenho do LabSystem.

2.3. Saúde digital e o portal do utente

A saúde digital integra, entre outros, os registos eletrónicos de saúde, a interoperabilidade e o acesso remoto à informação clínica, sendo a telemedicina a prestação remota de cuidados, síncrona ou assíncrona, apoiada em tecnologias digitais uma das suas componentes centrais [1]. A adoção destas soluções, fortemente acelerada pela pandemia de COVID-19, generalizou modelos de cuidados em que o doente acede aos seus dados à distância.

O portal do utente do LabSystem inscreve-se nesta tendência: permite que o paciente consulte os seus resultados, requisições e faturas em linha, sem deslocação, recebendo ainda notificações automáticas. Trata-se, assim, de um elemento de saúde digital que aproxima o laboratório do doente e prolonga o acompanhamento para fora das instalações físicas.

2.4. Tecnologias adotadas

Em coerência com os conteúdos da unidade curricular, o projeto adota JavaScript/TypeScript *full-stack*. O TypeScript [3], um superconjunto de JavaScript com tipagem forte e orientação a objetos, que compila para JavaScript e deteta erros em tempo de

compilação, é utilizado tanto no servidor como no cliente, conferindo robustez e legibilidade ao código.

No servidor recorre-se ao Node.js [4], plataforma de desenvolvimento de aplicações do lado do servidor em JavaScript, assente no motor V8 e num modelo de execução assíncrono e orientado a eventos, com gestão de dependências via NPM. A unidade curricular apresentou a *stack* MEAN (MongoDB, Express, Angular e Node.js); neste projeto adotou-se uma variante em que o React [8] substitui o Angular na camada de apresentação, uma biblioteca de componentes equivalente para a construção de interfaces. Assim, o servidor usa o *framework* Express [5] para expor a API, a persistência assenta no MongoDB [6], base de dados NoSQL orientada a documentos cuja flexibilidade se adequa à estrutura heterogénea dos dados clínicos (por exemplo, listas de tubos ou de análises), manipulada através do ODM Mongoose [7].

As restantes bibliotecas utilizadas, designadamente para autenticação, são introduzidas, quando relevante, nas secções de Arquitetura e de Segurança. A interoperabilidade em XML assenta no XML e no respetivo esquema XSD, validado no servidor com a biblioteca *xmlint-wasm* e processado com o *fast-xml-parser*.

3. Requisitos do Sistema

3.1. Requisitos Funcionais

A Tabela 1 enumera os requisitos funcionais (RF) identificados, agrupados por área funcional, com indicação dos perfis envolvidos.

Tabela 1 - Requisitos funcionais.

Ref.	Requisito funcional	Perfis
RF01	Autenticar utilizadores com email e palavra-passe, emitindo um token JWT.	Todos
RF02	Permitir a recuperação self-service da palavra-passe.	Todos
RF03	Permitir o registo de novas contas em estado «pendente de aprovação».	Público
RF04	Aprovar, ativar e desativar contas e atribuir/alterar papéis.	Administrador
RF05	Criar, pesquisar, editar e desativar fichas de utente.	Téc. / Méd. / Enf.
RF06	Atribuir um utente a um médico responsável.	Médico / Admin
RF07	Criar requisições analíticas a partir de um catálogo de análises.	Médico / Téc.
RF08	Definir prioridade da requisição (normal, urgente, STAT).	Médico / Téc.
RF09	Gerar amostras a partir de uma requisição e atribuir enfermeiro/técnico.	Enf. / Téc.
RF10	Registar colheita (presencial ou domiciliária), tubos e temperatura.	Enfermeiro
RF11	Rejeitar uma amostra com indicação de motivo.	Enf. / Téc.
RF12	Processar amostras recebidas e inserir resultados por análise.	Técnico
RF13	Calcular automaticamente a flag (normal/alto/baixo/crítico).	Sistema
RF14	Validar tecnicamente um resultado (1.ª assinatura).	Técnico
RF15	Validar clinicamente um resultado (2.ª assinatura) e emitir relatório.	Médico
RF16	Rejeitar um resultado com motivo.	Téc. / Méd.

RF17	Notificar resultados críticos e novos eventos (consultados por sondagem HTTP).	Sistema
RF18	Criar e gerir faturas (particular, SNS, seguradora) com comparticipação.	Administrador
RF19	Emitir, registar pagamento e anular faturas.	Administrador
RF20	Gerir equipamentos (estado, calibração) e manutenções.	Téc. / Admin
RF21	Gerir reagentes (stock, validade, alertas de mínimo).	Téc. / Admin
RF22	Apresentar indicadores e gráficos operacionais (analytics).	Gestão
RF23	Registar e consultar o histórico de ações.	Administrador
RF24	Disponibilizar ao utente um portal self-service (resultados, requisições, faturas).	Utente
RF25	Disponibilizar ao médico um portal com os seus utentes e validações.	Médico
RF26	Expor e configurar integrações externas (prescrição eletrónica, webhooks, exportação SAP).	Sistema

3.2. Requisitos Não Funcionais

Tabela 2 - Requisitos não funcionais.

Ref.	Categoria	Requisito não funcional
RNF01	Segurança	As palavras-passe são armazenadas cifradas com scrypt (função do módulo nativo de criptografia do Node.js), com salt aleatório por utilizador; a autenticação usa JWT com tokenVersion para invalidação forçada de sessões.
RNF02	Segurança	O endpoint de autenticação é protegido por um limitador de pedidos próprio (10 tentativas por 15 minutos, com resposta HTTP 429) e por bloqueio de conta após múltiplas falhas.
RNF03	Segurança	O acesso aos recursos é controlado por middleware de autorização baseado em papéis (RBAC).
RNF04	Privacidade	O sistema cumpre o RGPD, com registo de auditoria e minimização de dados pessoais.
RNF05	Desempenho	As listagens são paginadas e os campos de pesquisa indexados; a comunicação é assíncrona e não bloqueante.
RNF06	Escalabilidade	A arquitetura desacoplada (frontend, backend e base de dados separados) permite escalar o servidor de forma independente.
RNF07	Usabilidade	A interface é responsiva, consistente e orientada ao perfil do utilizador.
RNF08	Disponibilidade	Os dados são persistidos num volume dedicado; o healthcheck permite a monitorização do serviço.
RNF09	Manutenibilidade	O código adota TypeScript com tipagem forte e arquitetura em camadas, validado através de testes manuais da API.
RNF10	Portabilidade	O frontend é servido como conteúdo estático e a base de dados está alojada no MongoDB Atlas, simplificando a instalação.
RNF11	Integração	O sistema disponibiliza um contrato REST documentado e um esquema XML/XSD para troca com sistemas externos.
RNF12	Fiabilidade	As transições de estado são validadas no servidor, impedindo operações inválidas.

3.3. Casos de Uso

A Figura 1 sintetiza os principais casos de uso e a sua associação aos utilizadores, complementando a especificação dos requisitos funcionais.

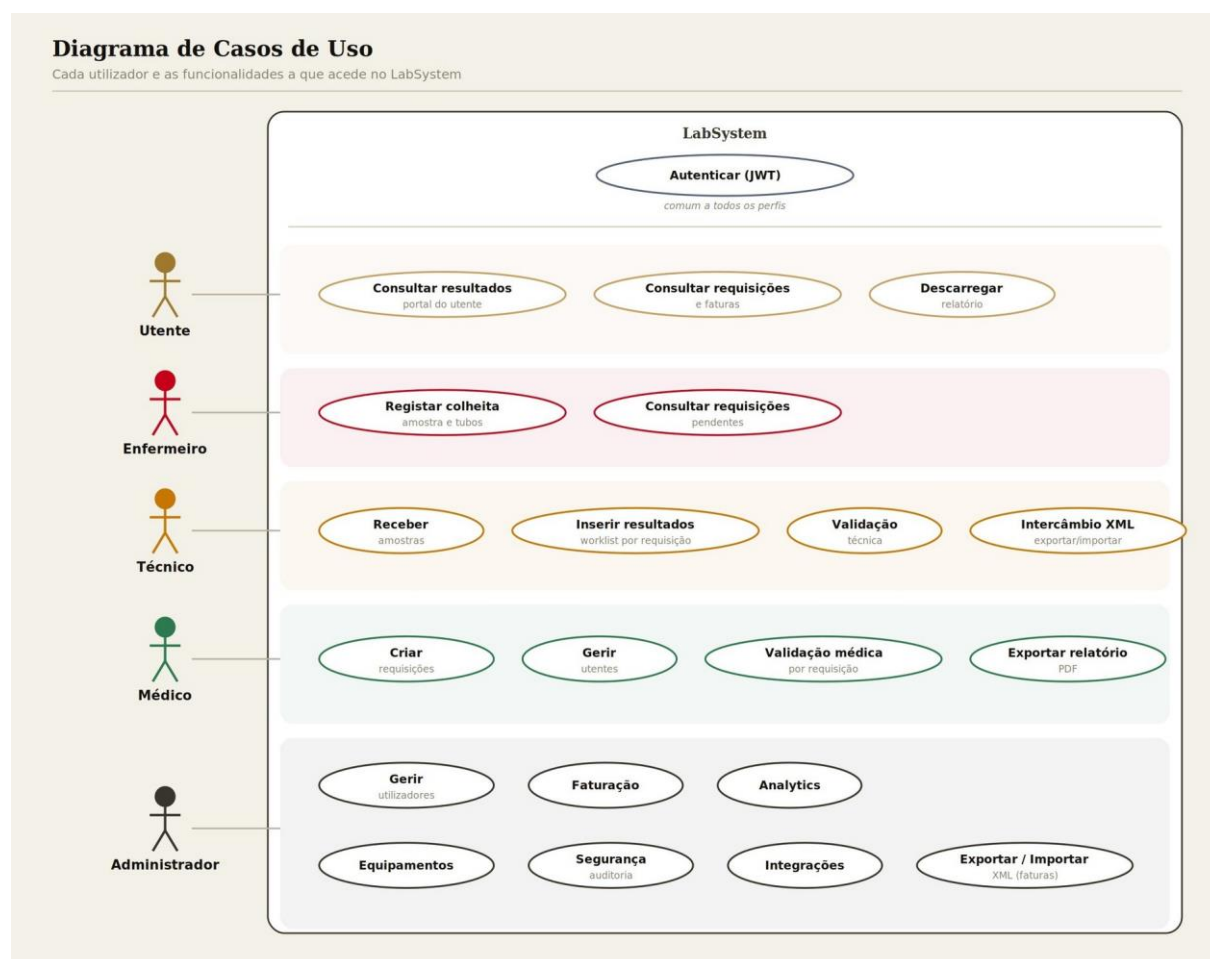


Figura 1 - Diagrama de casos de uso do LabSystem.

4. Arquitetura do Sistema

4.1. Visão geral

O LabSystem segue uma arquitetura cliente-servidor desacoplada. O cliente é uma aplicação de página única (SPA) em React que comunica com o servidor exclusivamente através de uma API REST (JSON). O servidor adota um padrão próximo do MVC adaptado a REST: os routers recebem os pedidos, o middleware trata da segurança e validação, os controllers concentram a lógica de negócio e os models (Mongoose) representam o domínio e a persistência. Esta separação clara de responsabilidades favorece a manutenibilidade e a testabilidade.

4.2. Diagrama de componentes

A Figura 2 identifica os componentes principais e as interfaces entre eles.

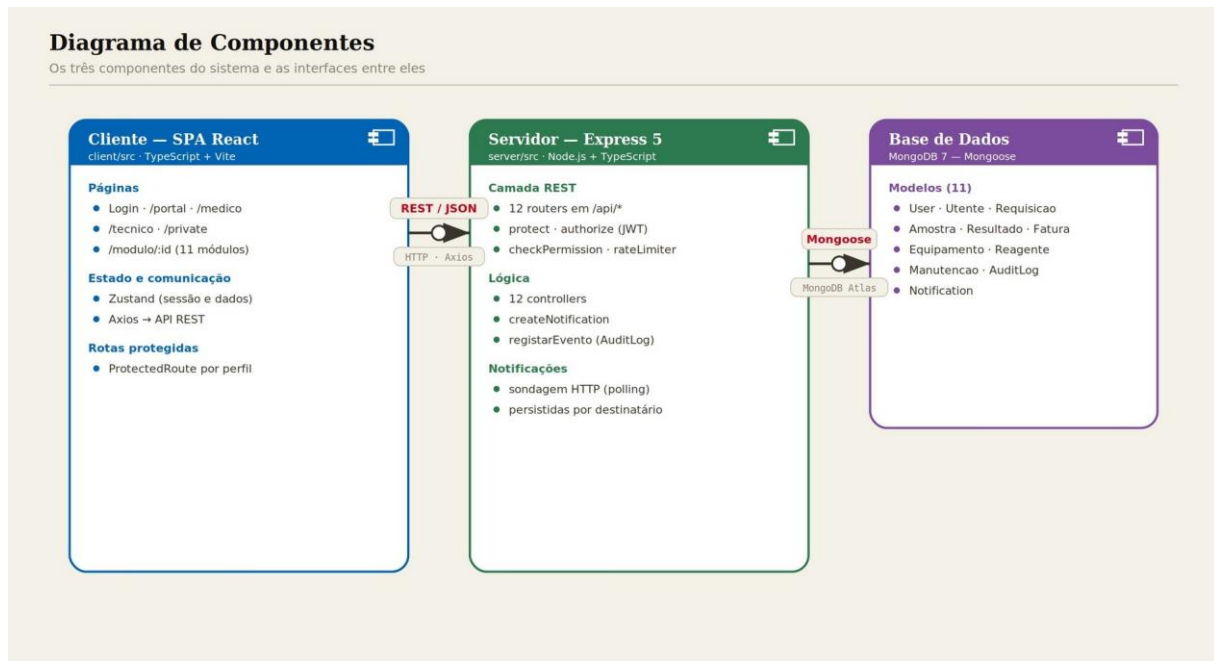


Figura 2 - Diagrama de componentes e interfaces de comunicação.

4.3. Vista Lógica

A vista lógica (Figura 3) organiza o software em quatro camadas — Apresentação, API, Domínio/Serviços e Acesso a Dados — identificando as pastas do código e as responsabilidades de cada camada.

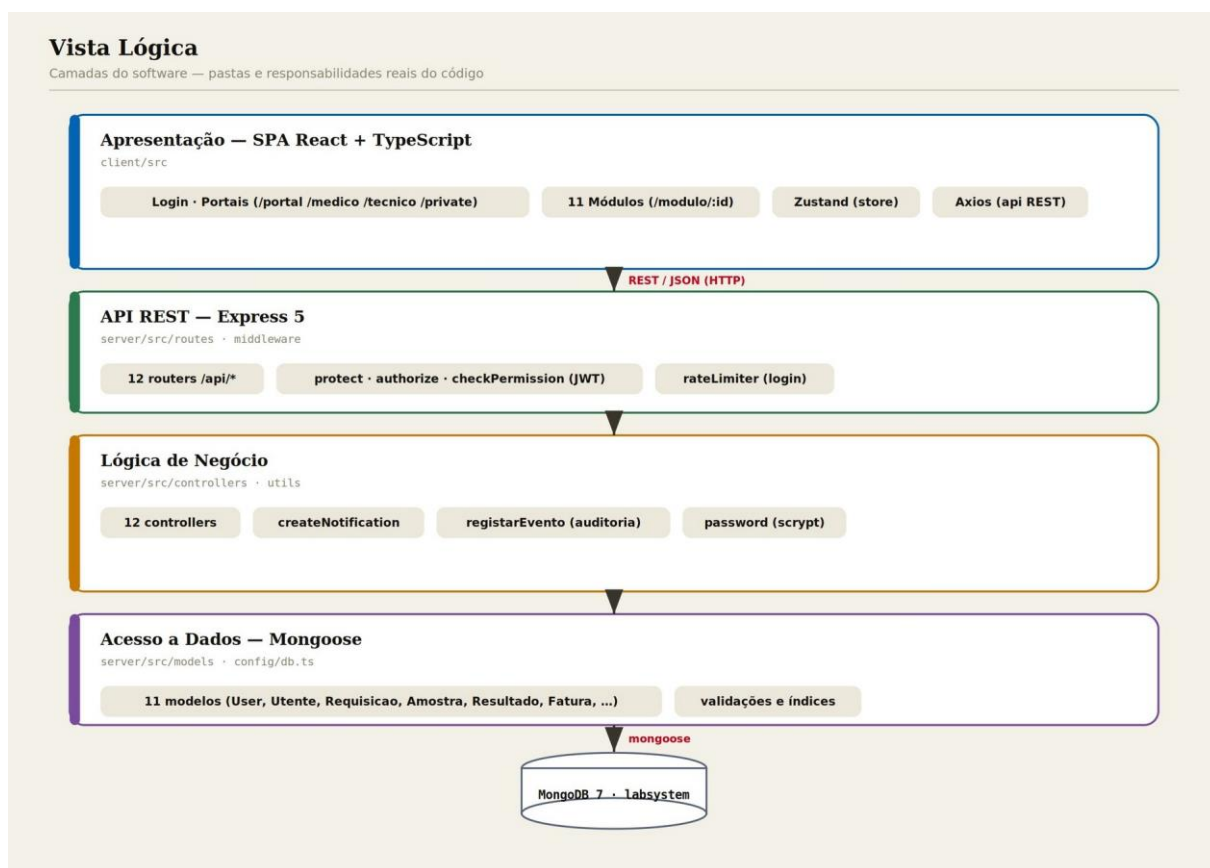


Figura 3 – Vista lógica: arquitetura em camadas do software.

4.4. Vista de Implantação

A implantação assenta em três componentes independentes (Figura 4), sem recurso a contentores. O frontend (React + Vite) é compilado e servido como conteúdo estático; o backend é uma API REST em Node.js + Express, exposta na porta 4000; e a base de dados é uma instância MongoDB alojada no serviço MongoDB Atlas (cloud). A comunicação faz-se exclusivamente por API REST (JSON), e os segredos de configuração - o JWT_SECRET, o MONGO_URI e a origem CORS autorizada - são fornecidos ao backend através de variáveis de ambiente.

Vista de Implantação

Três componentes ligados por API REST (JSON sobre HTTPS)



Comunicação exclusivamente por API REST (JSON). Sem contentores; segredos em variáveis de ambiente.

Figura 4 – Vista de implantação: três componentes ligados por API REST.

4.5. Integração e validação de dados em XML/XSD

Para garantir a interoperabilidade com sistemas externos designadamente a contabilidade, as seguradoras e outros laboratórios, o LabSystem implementa um ciclo completo de exportação, validação e importação de dados em XML. Embora os dados sejam persistidos internamente em JSON/BSON (MongoDB), o XML constitui um formato de troca normalizado, estruturado e amplamente suportado; a cada documento XML está associado um esquema XSD (XML Schema Definition) que define formalmente a sua estrutura e permite validá-lo automaticamente. O objetivo é duplo: assegurar a interoperabilidade e garantir a integridade dos dados, uma vez que qualquer documento é validado contra o respetivo esquema antes de ser gravado na base de dados.

O ciclo foi aplicado a três recursos do sistema, cada um com o seu esquema próprio: faturas (fatura.xsd), requisições (requisicao.xsd) e resultados (resultado.xsd). Cada esquema define um elemento raiz de coleção que contém vários elementos individuais e recorre a complexType com sequence para impor a ordem e a cardinalidade dos elementos (minOccurs/maxOccurs), bem como a simpleType com restriction e enumeration para restringir valores a conjuntos fechados, por exemplo, os estados válidos de uma fatura (rascunho, emitida, paga, anulada) ou de um resultado. Os tipos de dados utilizados incluem xs:string, xs:date, xs:dateTime, xs:decimal e xs:boolean.

O fluxo de funcionamento é idêntico nos três recursos. Na exportação, o backend gera um documento XML com os registos, respeitando a ordem definida no esquema e aplicando o escape correto de caracteres especiais (por exemplo, & é convertido em &); o ficheiro é descarregado pelo utilizador. Na importação, o servidor valida primeiro o documento submetido contra o respetivo XSD: se for inválido, devolve um erro HTTP 422 com a lista de erros (mensagem e número de linha de cada um) e não grava absolutamente nada, protegendo a base de dados; só quando o XML é válido é que os dados são importados, atualizando os registos existentes identificados pelo número ou código. A funcionalidade expõe seis endpoints REST protegidos por JWT, GET e POST de /faturas, /requisicoes e /resultados em /export/xml e /import/xml, estando a importação restrita aos perfis de administrador e técnico. A validação recorre à biblioteca xmllint-wasm (o motor libxml2 compilado para WebAssembly, executado no servidor) e o parsing do XML importado é feito com o fast-

xml-parser. O fluxo de importação com validação está representado na Figura 30; os esquemas XSD completos e exemplos de instâncias XML (incluindo um caso inválido e a respetiva rejeição) encontram-se nos Anexos A e B.

5. Modelo de Domínio

O modelo de domínio (Figura 5) captura as entidades de negócio e as suas associações. A entidade central é a Requisição, que agrega análises pedidas para um Utente; cada requisição origina uma ou mais Amostras, das quais resultam Resultados analíticos sujeitos a dupla

Diagrama de Classes do Domínio

Entidades centrais (circuito laboratorial) e entidades de apoio

```

classDiagram
    class User {
        + nome: string
        + email: string (U)
        + role: UserRole
        + ativo: boolean
        + loginAttempts: number
        + tokenVersion: number
        + matchPassword(p): bool
        + hasPermission(n,a): bool
        + isLocked(): bool
    }
    class Utente {
        + numeroProcesso: string (U)
        + nome: string
        + nif: string (U)
        + sms: string (U)
        + genero: Genero
        + ativo: boolean
    }
    class Equipamento {
        + nome: string
        + numeroSerie: string
        + estado: EstadoEquipamento
        + proximoCalibragem: Date
    }
    class Manutencao {
        + tipo: TipoManutencao
        + descricao: string
        + data: Date
        + resolvido: boolean
    }
    class Reagente {
        + nome / codigo / lote: string
        + validade: Date
        + quantidadeAtual: number
        + quantidadeFinal: number
    }
    class Amostra {
        + codigoAmostra: string (U)
        + tipoColheita: TipoColheita
        + dataHoraColheita: Date
        + temperatura: number
        + estado: EstadoAmostra
    }
    class Resultado {
        + codigoResultado: string (U)
        + valor: string + unidade?: string
        + flag: FlagResultado
        + estado: EstadoResultado
        + relatorioEmiteido: boolean
    }
    class Assinatura {
        + nome: string
        + datahora: Date
        + observacoes?: string
    }
    class Fatura {
        + numeroFatura: string (U)
        + tipo: TipoFatura
        + valorBruto / valorLiquido: number
        + percentCompensacao: number
        + estado: EstadoFatura
    }
    class Rejeicao {
        + nome: string
        + datahora: Date
        + motivo: string
    }
    class LinhaFatura {
        + codigo: string
        + descricao: string
        + preco: number
    }
    class AuditLog {
        + utilizador: string
        + acao / modulo: string
        + createdAt: Date
    }
    class Morada {
        + rua: string
        + codigoPostal: string
        + localidade: string
    }
    class Tubo {
        + tipo: TipoTubo
        + analises: string[]
        + coletado: boolean
    }
    class Analise {
        + codigo: string
        + nome: string
        + categoria: string
    }
    class Notification {
        + tipo: TipoNotificacao
        + titulo: string
        + lida: boolean
    }

    User "0..*" -- "0" Utente : utentidado
    User "0..1" -- "0" Utente : numeroProcesso
    User "1" -- "0" Requisicao : requisicaoId
    User "1" -- "0" Requisicao : requisicaoId
    Utente "0" -- "1" Requisicao
    Requisicao "0..1" -- "1" Amostra
    Amostra "0..1" -- "1" Resultado
    Resultado "0..1" -- "0..1" Assinatura
    Assinatura "0..1" -- "0..1" Resultado
    Resultado "0..1" -- "0..1" Fatura
    Fatura "0..1" -- "0..1" Rejeicao
    Rejeicao "0..1" -- "0..1" LinhaFatura
    LinhaFatura "0..1" -- "0..1" Fatura
    LinhaFatura "0..1" -- "0..1" AuditLog
    AuditLog "0..1" -- "0..1" Fatura
    AuditLog "0..1" -- "0..1" LinhaFatura
    Morada "1" -- "0..1" Amostra
    Morada "1" -- "0..1" Resultado
    Morada "1" -- "0..1" Fatura
    Morada "1" -- "0..1" LinhaFatura
    Tubo "1" -- "0..1" Amostra
    Tubo "1" -- "0..1" Resultado
    Tubo "1" -- "0..1" Fatura
    Tubo "1" -- "0..1" LinhaFatura
    Analise "1" -- "0..1" Amostra
    Analise "1" -- "0..1" Resultado
    Analise "1" -- "0..1" Fatura
    Analise "1" -- "0..1" LinhaFatura
    Notification "1" -- "0..1" Amostra
    Notification "1" -- "0..1" Resultado
    Notification "1" -- "0..1" Fatura
    Notification "1" -- "0..1" LinhaFatura
  
```

Associação → Composição ◆-1 (U) chave única / ? campo opcional

Seguidamente descreve-se cada entidade, com a respetiva estrutura de dados (campo, tipo e descrição), tal como implementada nos modelos Mongoose.

Representa qualquer interveniente autenticado do sistema, com o seu papel e estado de segurança.

Campo	Tipo	Descrição
nome	String	Nome completo do utilizador.
email	String (único)	Identificador de autenticação.
password	String (hash)	Palavra-passe protegida com scrypt (módulo nativo do Node), no formato salt:hash.
role	Enum	administrador, técnico, médico, enfermeiro ou utente.
ativo	Boolean	Indica se a conta está ativa/aprovada.

avatar, telefone, departamento	String	Informação de perfil complementar.
ultimoLogin	Date	Data/hora do último início de sessão.
utenteRef	ObjectId → Utente	Liga uma conta de utente à respetiva ficha clínica.
loginAttempts, lockUntil	Number / Date	Suportam o bloqueio de conta por tentativas falhadas.
tokenVersion	Number	Permite a invalidação forçada de tokens JWT.

5.2. Utente (paciente)

Tabela 4 - Entidade Utente.

Campo	Tipo	Descrição
numeroProcesso	String (único)	Número de processo clínico.
nome, dataNascimento, genero	String / Date	Dados demográficos do paciente.
nif	String (único)	Número de identificação fiscal.
sns	String (único)	Número de utente do SNS.
contacto, email	String	Dados de contacto.
morada	Objeto	Rua, código postal e localidade.
medicoId / medicoNome	ObjectId → User / String	Médico responsável atribuído.
observacoes	String	Notas clínicas livres.
ativo	Boolean	Estado do registo.

5.3. Requisição

Tabela 5 - Entidade Requisicao.

Campo	Tipo	Descrição
numeroRequisicao	String (único)	Identificador da requisição.
utente / utenteNome / utenteProcesso	ObjectId → Utente / String	Utente associado (com desnormalização).
medicoSolicitante	String	Médico que solicitou as análises.
analises	Array<{codigo, nome, categoria}>	Conjunto de análises pedidas.
urgente / prioridade	Boolean / Enum	normal, urgente ou stat.
estado	Enum	pendente, em_curso, concluida ou cancelada.
prescricaoRef	String	Referência da prescrição eletrónica.
observacoes / createdBy	String / ObjectId → User	Notas e autor do registo.

Tabela 5 - Entidade Requisicao.

5.4. Amostra

Tabela 6 - Entidade Amostra.

Campo	Tipo	Descrição
codigoAmostra	String (único)	Identificador da amostra.
requisicao / utente	ObjectId	Referências à requisição e ao utente.
tubos	Array<{tipo, analises[], coletado}>	Tubos (edta, citrato, gel, heparina, urina, outro).
tipoColheita	Enum	presencial ou domiciliaria.
moradaColheita	String	Morada, no caso de colheita domiciliária.
dataHoraColheita / temperatura	Date / Number	Momento da colheita e temperatura de transporte.
estado	Enum	aguarda_colheita, colhida, em_transito, recebida, rejeitada.
enfermeiroAtribuido / tecnicoAtribuido	ObjectId → User	Profissionais responsáveis.
motivoRejeicao	String	Motivo, em caso de rejeição.

Tabela 6 - Entidade Amostra.

5.5. Resultado

Tabela 7 - Entidade Resultado.

Campo	Tipo	Descrição
codigoResultado	String (único)	Identificador do resultado.
amostra / requisicao / utente	ObjectId	Referências de rastreabilidade.
analise	Objeto {codigo, nome, categoria}	Análise a que respeita o resultado.
equipamento, valor, unidade	String	Equipamento usado e valor medido.
refMin / refMax	Number	Intervalo de referência.
flag	Enum	pendente, normal, alto, baixo, critico_alto, critico_baixo.
estado	Enum	pendente ... validado_tecnico, validado_medico, rejeitado.
validacaoTecnica / validacaoMedica	Objeto {userId, nome, dataHora, obs}	Assinaturas digitais das validações.
rejeicao	Objeto {userId, nome, dataHora, motivo}	Registo de rejeição.
relatorioEmitido	Boolean	Indica se o relatório foi emitido.

5.6. Fatura

Tabela 8 - Entidade Fatura.

Campo	Tipo	Descrição
numeroFatura	String (único)	Identificador da fatura.
requisicao / utente	ObjectId	Referências associadas.
tipo / seguradora	Enum / String	particular, sns ou seguradora.
linhas	Array<{codigo, descricao, preco}>	Linhas de faturação.
valorBruto / percentComparticipacao	Number	Valor bruto e percentagem participada.
valorComparticipado / valorLiquido	Number	Valores calculados.
estado	Enum	rascunho, emitida, paga, anulada.
referenciaPagamento / dataEmissao / dataPagamento	String / Date	Dados de pagamento.

5.7. Equipamento, Reagente e Manutencao

Estas entidades suportam a gestão de recursos do laboratório.

Tabela 9 - Entidades Equipamento, Reagente e Manutencao.

Campo	Tipo	Descrição
Equipamento.nome / tipo / numeroSerie	String	Identificação do equipamento.
Equipamento.estado	Enum	operacional, manutencao, avariado, calibracao.
Equipamento.dataUltimaCalibração / proximaCalibração	Date	Controlo de calibrações.
Reagente.lote / validade	String / Date	Lote e validade do reagente.
Reagente.quantidade Atual / quantidadeMinima	Number	Suportam alertas de stock mínimo.
Manutencao.tipo / resolvido	Enum / Boolean	preventiva ou corretiva; estado de resolução.

5.8. AuditLog e Notification

A entidade AuditLog regista de forma imutável os eventos sensíveis (utilizador, ação, módulo, entidade e timestamp), suportando a conformidade com o RGPD. A entidade Notification suporta as notificações entregues a cada utilizador: cada notificação é persistida

e dirigida ao respetivo destinatário, sendo consultada pelo cliente através de sondagem periódica à API (tipo, título, ligação, estado de leitura e destinatário).

6. Fluxos de Trabalho Clínicos

6.1. Fluxo completo

O processo clínico-laboratorial é estritamente sequencial e auditado. A Figura 6 ilustra o ciclo de vida de uma análise, desde a requisição até à faturação, com a indicação dos utilizadores e dos estados em cada etapa. O médico cria a requisição; o enfermeiro realiza a colheita; o técnico processa a worklist, insere o resultado e efetua a validação técnica; o médico procede à validação médica e emite o relatório; e, por fim, o administrador (que assumiu as funções financeiras) emite a fatura.



Figura 6 – Fluxo clínico completo e máquina de estados.

6.2. Diagrama de estados da Amostra

A amostra evolui ao longo dos estados apresentados na Figura 7, podendo ser rejeitada com registo de motivo a partir de qualquer estado intermédio.

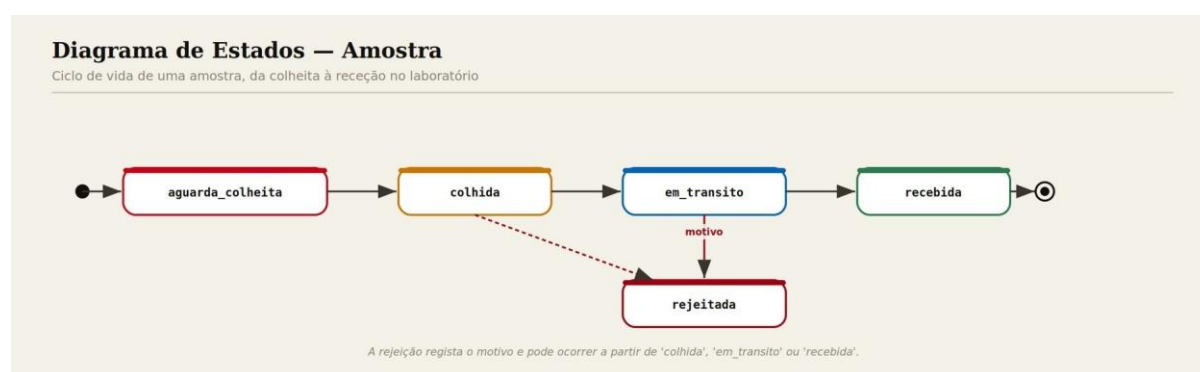


Figura 7 – Diagrama de estados da Amostra.

6.3. Diagrama de estados do Resultado

O resultado percorre uma sequência que culmina na dupla validação (Figura 8). A *flag* é calculada automaticamente a partir dos valores de referência; a validação técnica constitui a primeira assinatura e a validação médica a segunda, desencadeando a emissão do relatório.

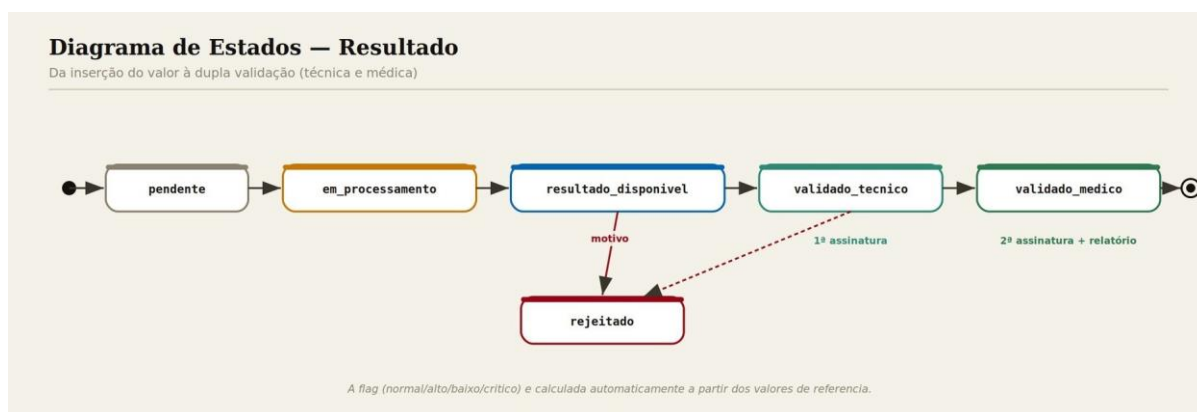


Figura 8 – Diagrama de estados do Resultado.

6.4. Diagrama de estados da Fatura

A fatura evolui de rascunho a paga, podendo ser anulada (Figura 9). O valor líquido resulta da aplicação da comparticipação ao valor bruto, consoante o tipo de faturação.

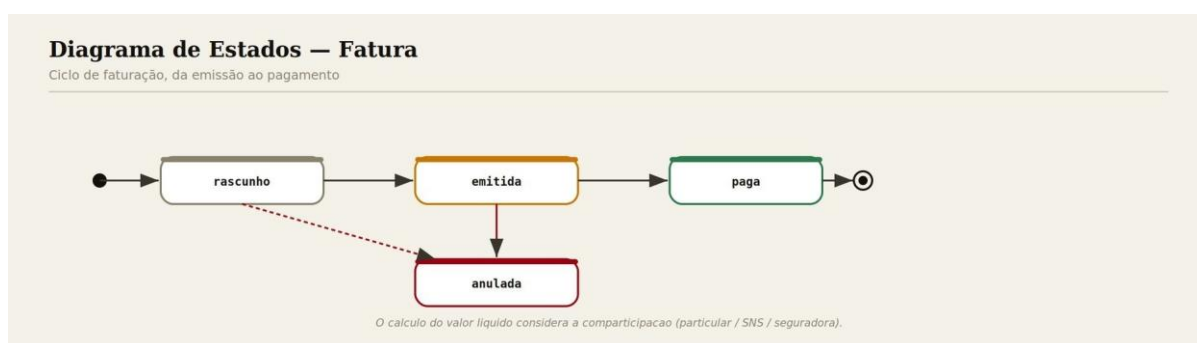


Figura 9 – Diagrama de estados da Fatura.

6.5. Atribuição Médico–Utente

Cada utente pode ter um médico responsável atribuído (campo `medicoId`). Esta associação determina o conjunto de utentes e de requisições visíveis no Portal Médico e suporta o encaminhamento de validações e notificações para o profissional adequado.

7. Segurança e Controlo de Acessos

7.1. Modelo RBAC

O acesso às funcionalidades é regido por um modelo de controlo de acessos baseado em papéis (RBAC). A Tabela 10 resume as permissões por perfil (C = criar, R = consultar, U = atualizar, D = eliminar).

Tabela 10 - Matriz de permissões (RBAC).

Perfil	Permissões por módulo
Administrador	Todos os módulos, com CRUD completo.
Técnico	Utentes (R); Requisições (CRU); Colheita (CRU); Análise (CRU); Validação (RU); Equipamentos (CRU); Relatórios (R); Analytics (R).
Médico	Utentes (CRU); Requisições (CRU); Colheita (R); Análise (R); Validação (CRU); Relatórios (CRU); Analytics (R).

Enfermeiro	Utentes (RU); Requisições (CR); Colheita (CRU).
Perfil	Permissões por módulo
Utente	Acesso apenas aos próprios dados: Utentes (R), Requisições (R), Relatórios (R), Financeiro (R).

Tabela 10 - Matriz de permissões (RBAC).

7.2. Autenticação JWT e versionamento de token

A autenticação assenta em JSON Web Tokens [9] emitidos no início de sessão e verificados em cada pedido pelo *middleware* `verifyToken`. Cada utilizador possui um campo `tokenVersion`; sempre que é necessário invalidar sessões (por exemplo, após reposição de palavra-passe ou desativação da conta), incrementa-se este valor, tornando inválidos todos os tokens anteriormente emitidos.

7.3. Proteção das palavras-passe e do login

As palavras-passe nunca são guardadas em claro: são protegidas com *bcrypt*, uma função de derivação de chave do módulo nativo de criptografia do Node.js, com um *salt* aleatório por utilizador, e comparadas em tempo constante. O endpoint de autenticação é ainda protegido por um limitador de pedidos implementado como *middleware* Express próprio (sem dependências externas), que admite até dez tentativas por janela de quinze minutos, respondendo com o código 429 quando excedido. Adicionalmente, o sistema contabiliza as tentativas falhadas por conta (`loginAttempts`) e, ao exceder o limite, aplica um bloqueio temporário (`lockUntil`), mitigando ataques de força bruta. As mensagens de erro são genéricas, de modo a não revelar a existência de contas.

7.4. Histórico de ações no sistema

Todas as ações sensíveis são gravadas na entidade *AuditLog*, com indicação do utilizador, da ação, do módulo e do momento. Este registo é consultável no módulo de Segurança e constitui um elemento essencial de conformidade com o RGPD e de rastreabilidade.

8. Interface Gráfica - Demonstração das Funcionalidades

Esta secção ilustra as principais funcionalidades através de capturas de ecrã da aplicação em funcionamento. A interface adota uma linguagem visual própria, com tipografia *display* serifada, uma paleta clínica e um código de cores por módulo.

8.1. Ecrã inicial do Administrador

Após a autenticação, o administrador é recebido pelo ecrã inicial do sistema (Figura 10), que apresenta a identidade visual do LabSystem e o mote institucional «Análises que falam contigo». Este ecrã não é uma página pública, mas sim o ponto de entrada do perfil de administração: a partir do menu de navegação, o administrador acede aos onze módulos operacionais do sistema.



Figura 10 – Ecrã inicial do administrador após autenticação.

8.2. Página de Login

O ecrã de autenticação (Figura 11) recebe as credenciais e disponibiliza acessos rápidos de demonstração para cada perfil, estando protegido por limitação de taxa e bloqueio de conta.

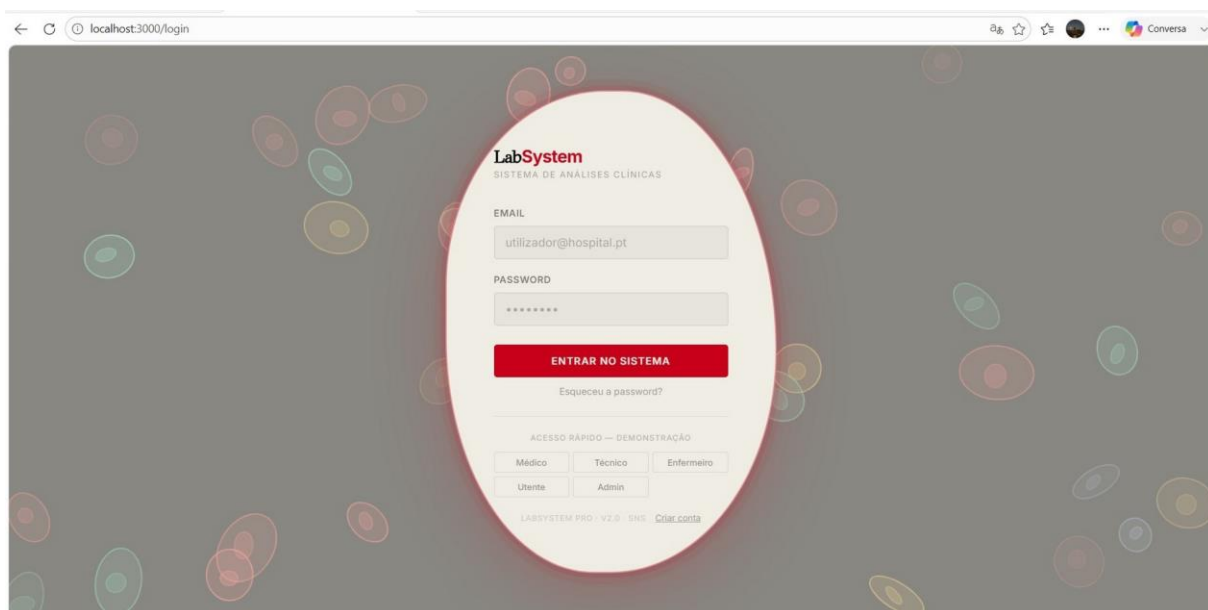


Figura 11 – Página de login.

8.3. Recuperação de Password

O fluxo self-service de recuperação de palavra-passe permite ao utilizador redefinir o acesso mediante confirmação da sua identidade (Figura 12).

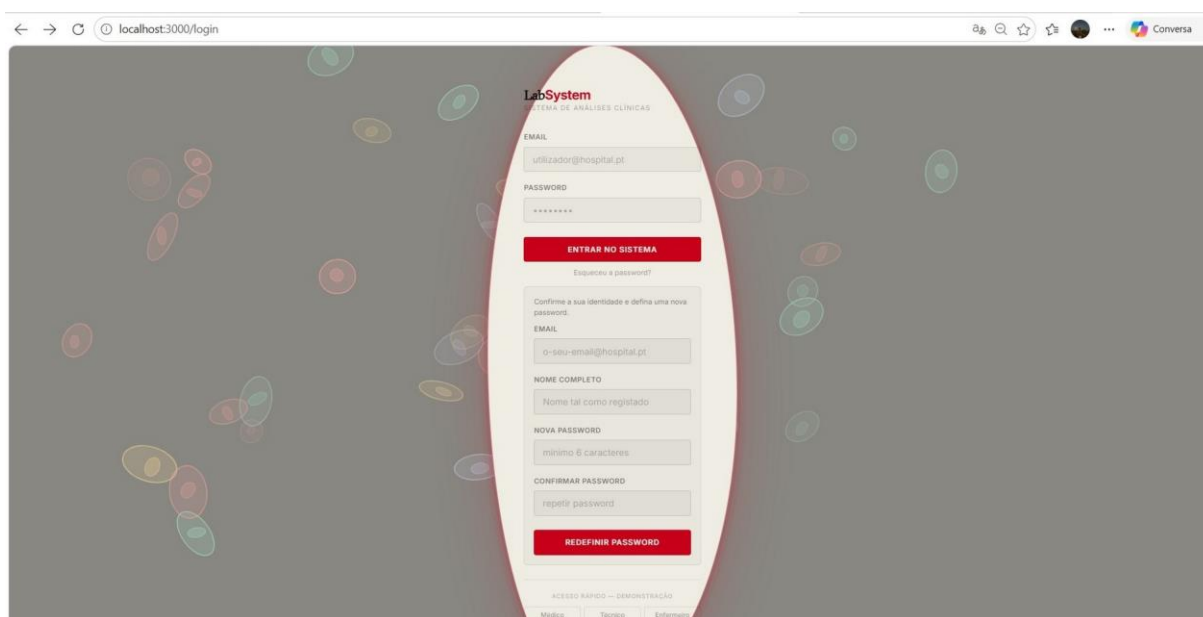


Figura 12 – Recuperação de palavra-passe.

8.4. Registo / Criação de Conta

O formulário de criação de conta (Figura 13) submete um pedido que fica «pendente de aprovação» pelo administrador, garantindo o controlo de acessos.

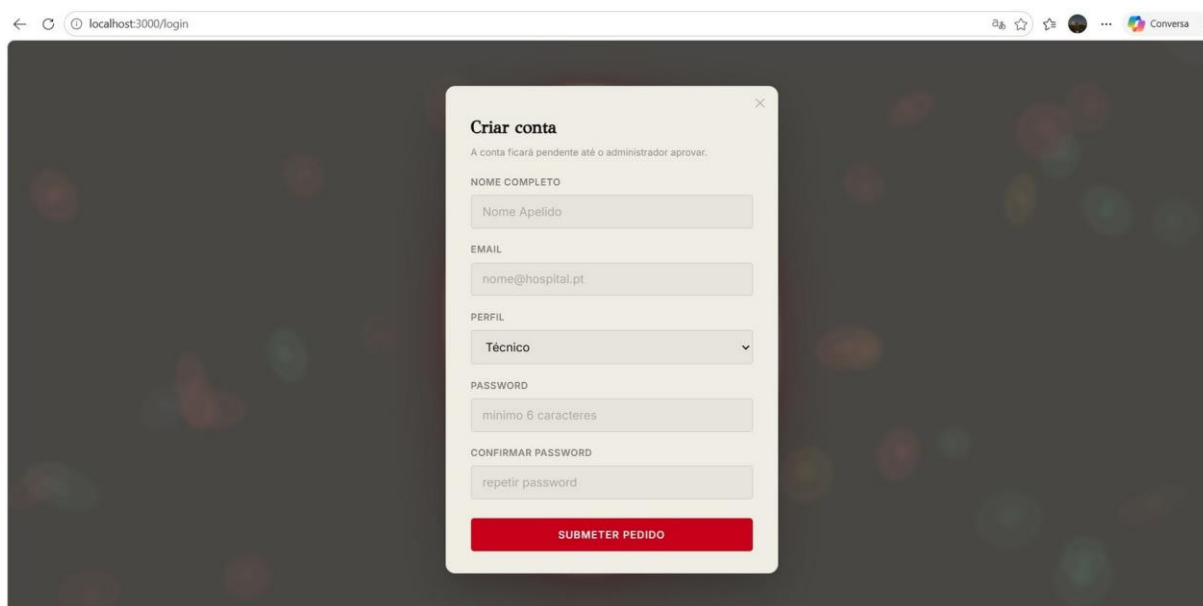


Figura 13 – Criação de conta (pendente de aprovação).

8.5. Portal do Utente

O portal do utente (Figura 14) reúne os seus resultados, requisições e faturas, num único local. O utente pode ainda descarregar o relatório validado em formato PDF.

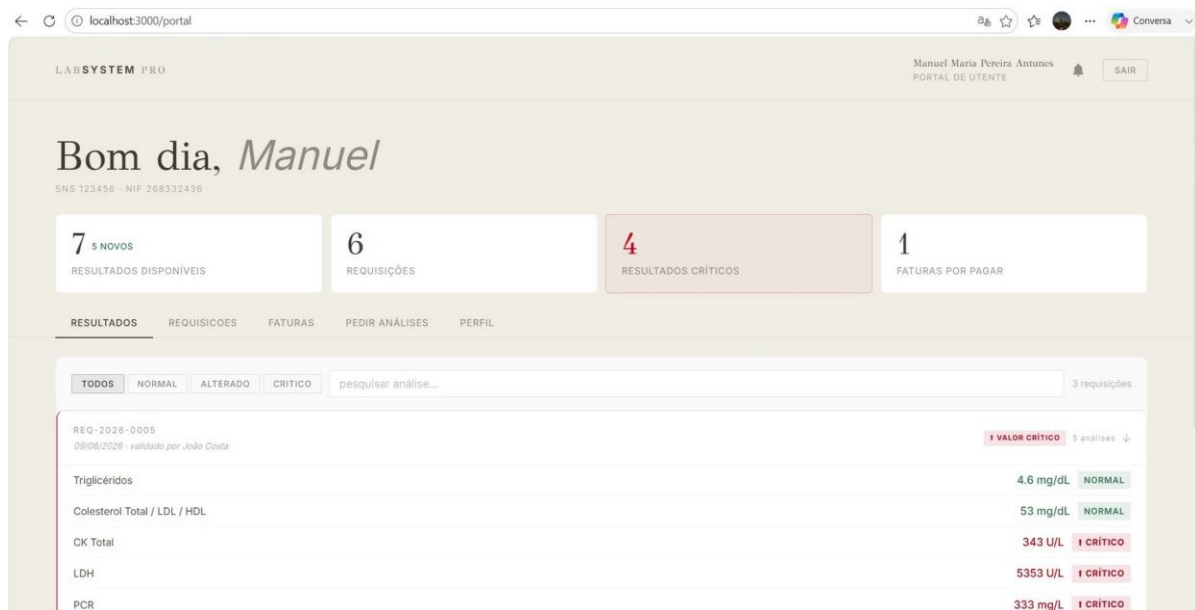


Figura 14 – Portal do utente.

8.6. Portal Médico

O portal médico (Figura 15) apresenta os utentes atribuídos, as requisições e os resultados que aguardam validação clínica. A validação médica é efetuada em bloco, por requisição, e o relatório clínico validado pode ser exportado em PDF.

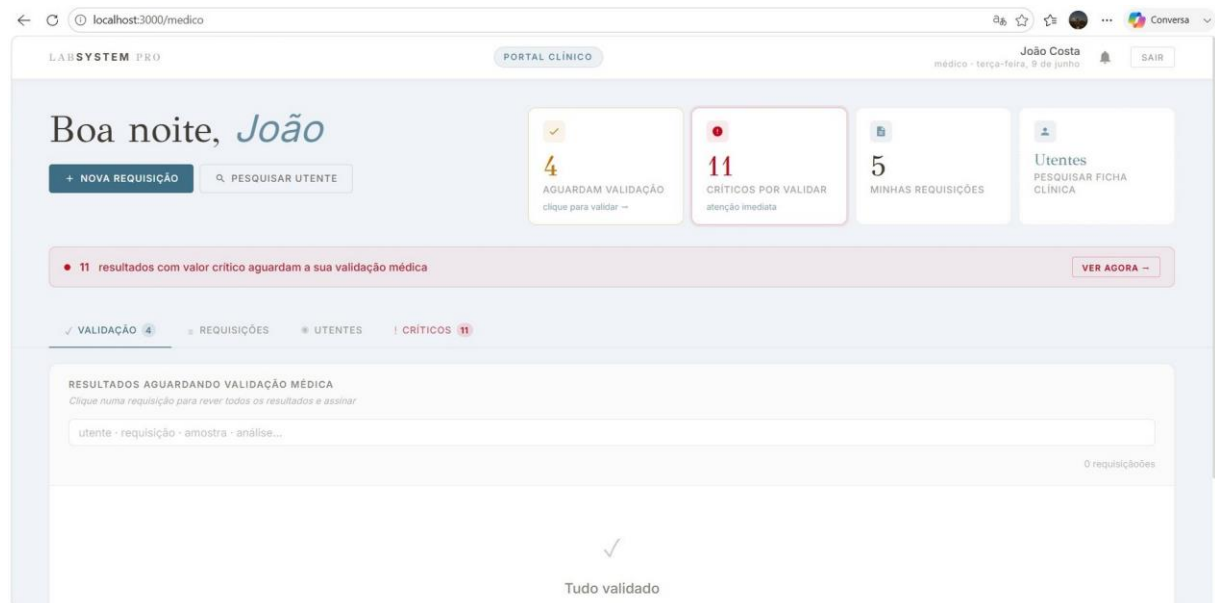


Figura 15 – Portal médico.

8.7. Portal Técnico

O portal do técnico (Figura 16) concentra a receção de amostras e o acompanhamento das análises e validações pendentes. A worklist encontra-se agrupada por requisição, permitindo inserir de uma só vez todos os resultados de um pedido.

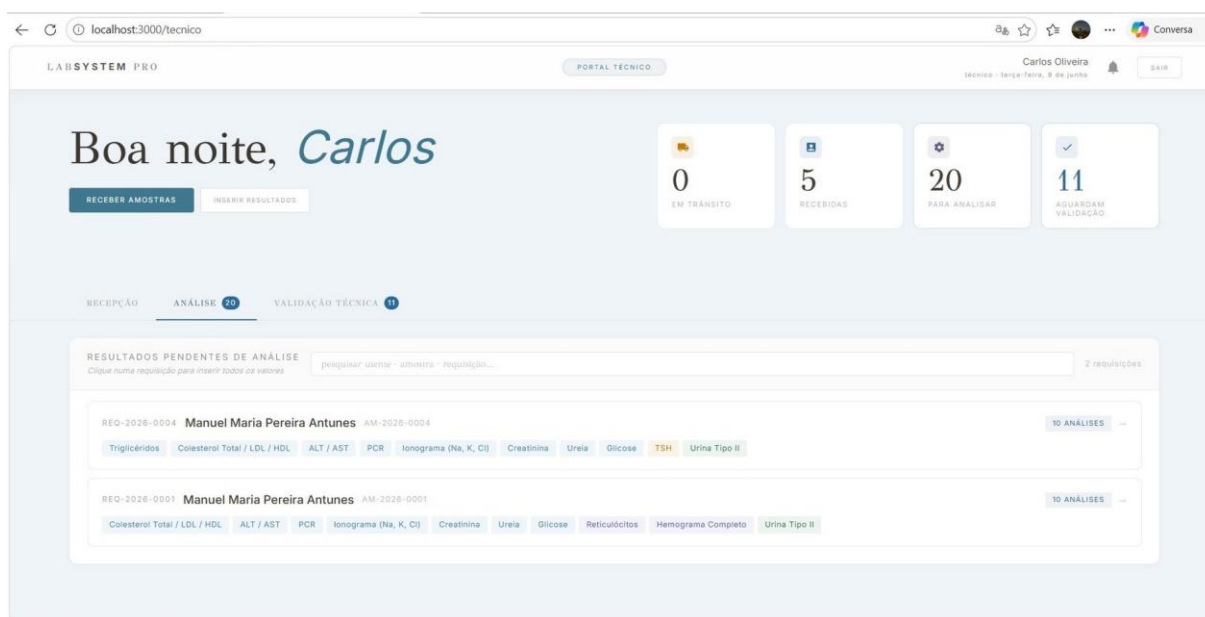


Figura 16 – Portal técnico.

8.8. Módulo Utentes

O módulo de utentes (Figura 17) permite a listagem, pesquisa, criação e consulta da ficha clínica.

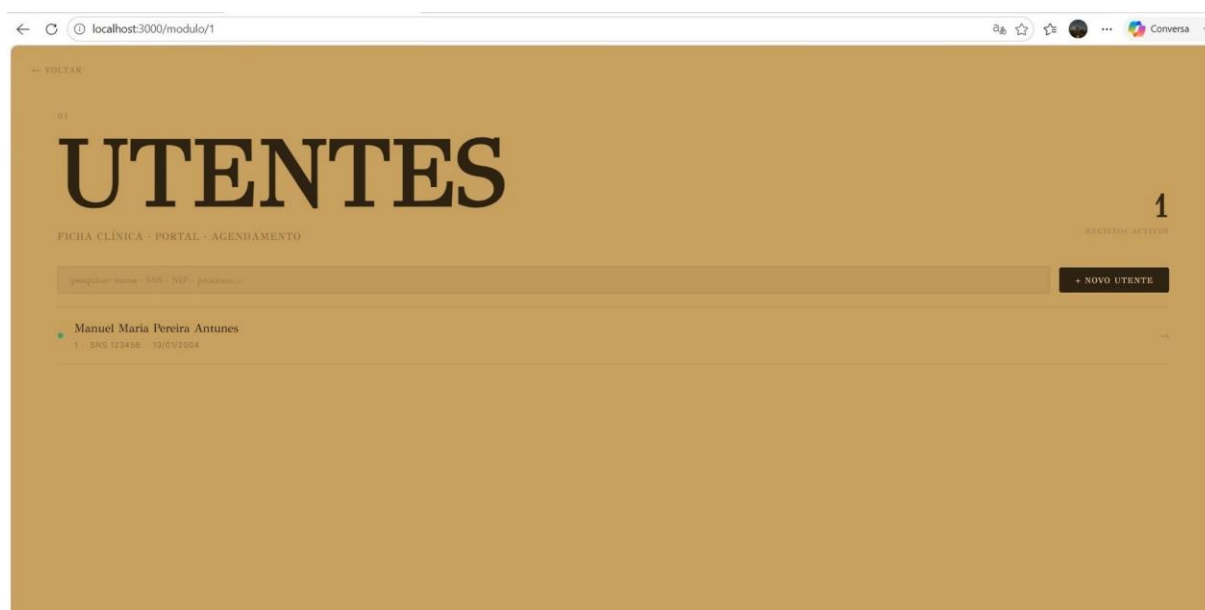


Figura 17 – Módulo de utentes.

8.9. Módulo Requisições

O módulo de requisições (Figura 18) suporta a criação de pedidos analíticos e a sua gestão por estado e prioridade.

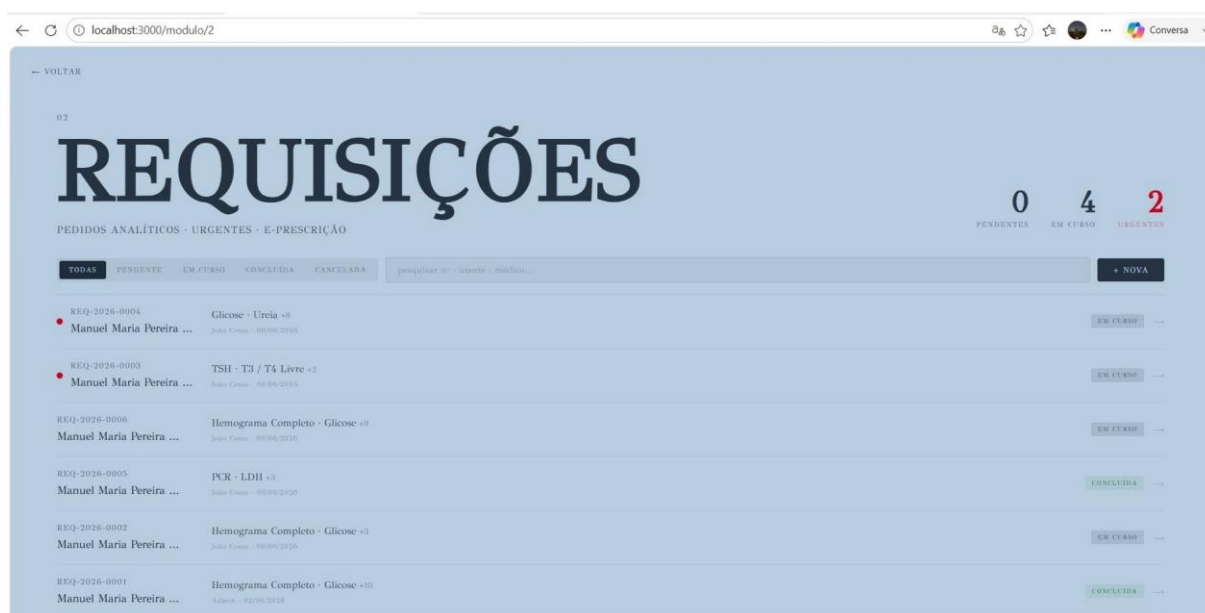


Figura 18 – Módulo de requisições.

8.10. Módulo Colheita

O módulo de colheita (Figura 19) gere as amostras, os tubos e a sua atribuição, com registo do estado e suporte a colheita domiciliária.



Figura 19 – Módulo de colheita.

No portal de enfermagem (Figura 20), o enfermeiro regista a colheita e gera a guia de tubos.

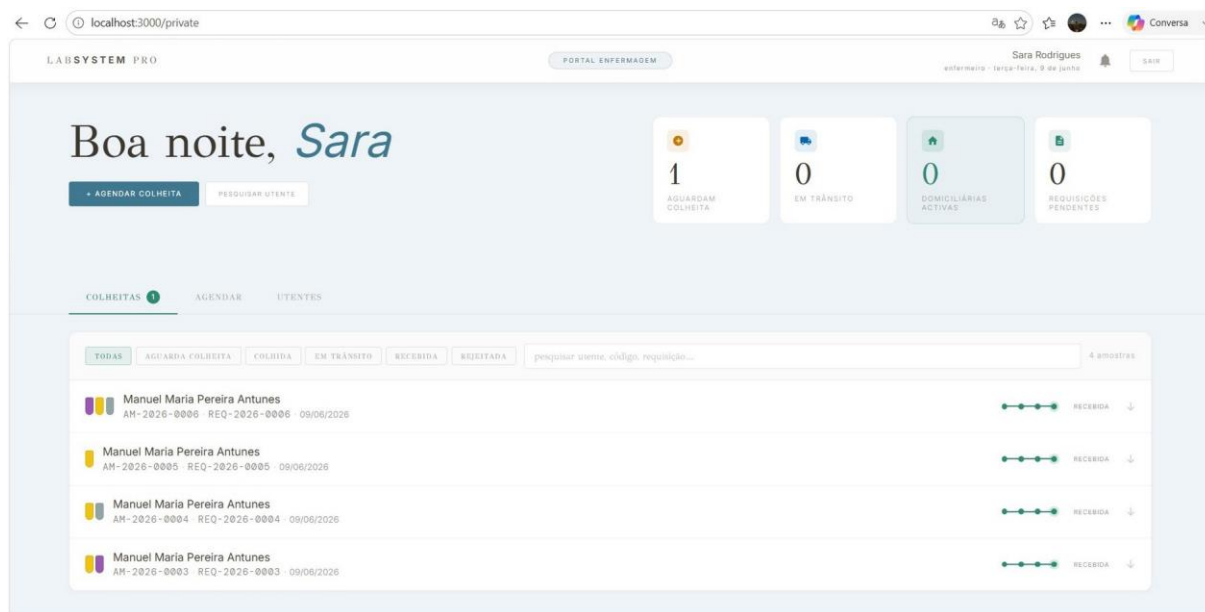


Figura 20 – Portal de enfermagem — registo de colheita.

8.11. Módulo Análise

O módulo de análise (Figura 21) apresenta a worklist de amostras recebidas, por categoria.

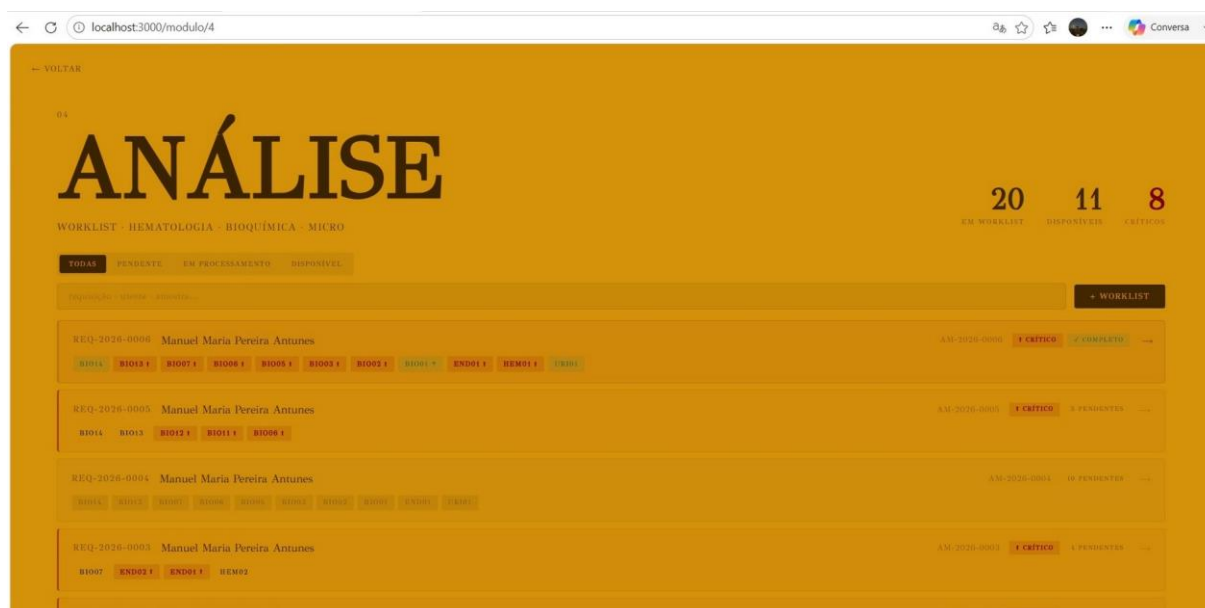


Figura 21 – Módulo de análise — worklist.

A inserção do resultado calcula automaticamente a flag a partir dos valores de referência; a validação técnica é depois realizada no módulo de validação (Figura 22).

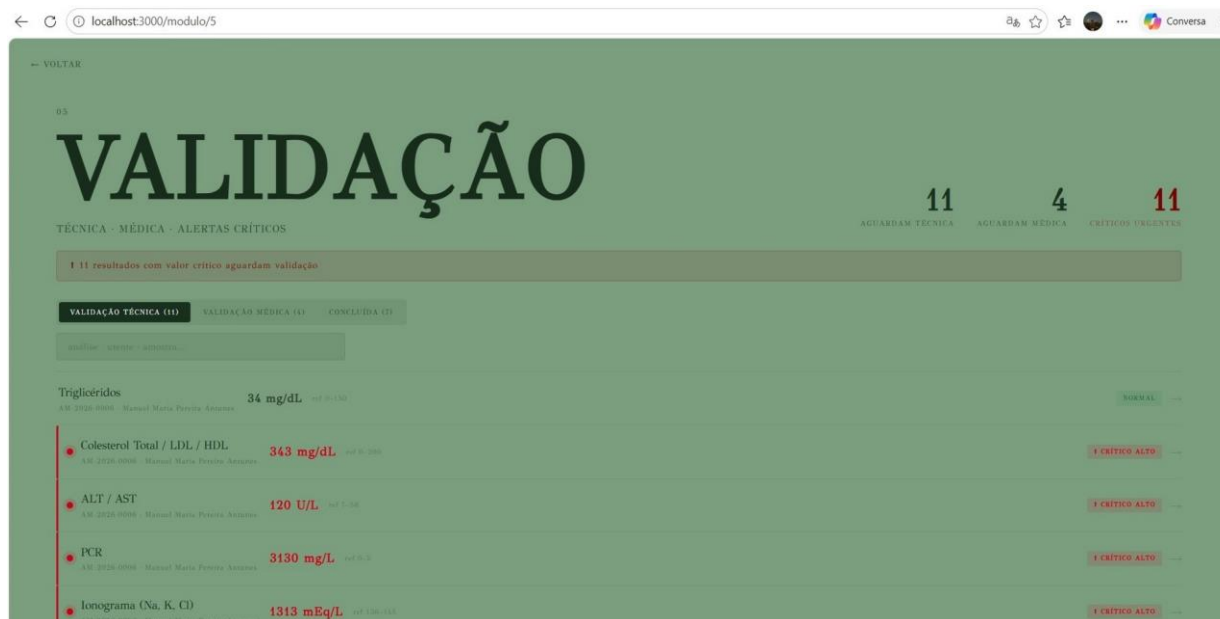


Figura 22 – Módulo de validação.

8.12. Módulo Validação

A validação distingue a etapa técnica da médica, registrando as duas assinaturas digitais. A validação médica, ilustrada no portal médico (Figura 15), conclui o processo e desencadeia a emissão do relatório, com notificação imediata em caso de valor crítico. Com a organização do fluxo por requisição, a validação processa em bloco todos os resultados de um pedido.

8.13. Módulo Financeiro

O módulo financeiro (Figura 23) suporta a criação e gestão de faturas, distinguindo particular, SNS e seguradora, e calculando a comparticipação. Com a integração do perfil financeiro no de administrador, este módulo passou a ser gerido pelo administrador.

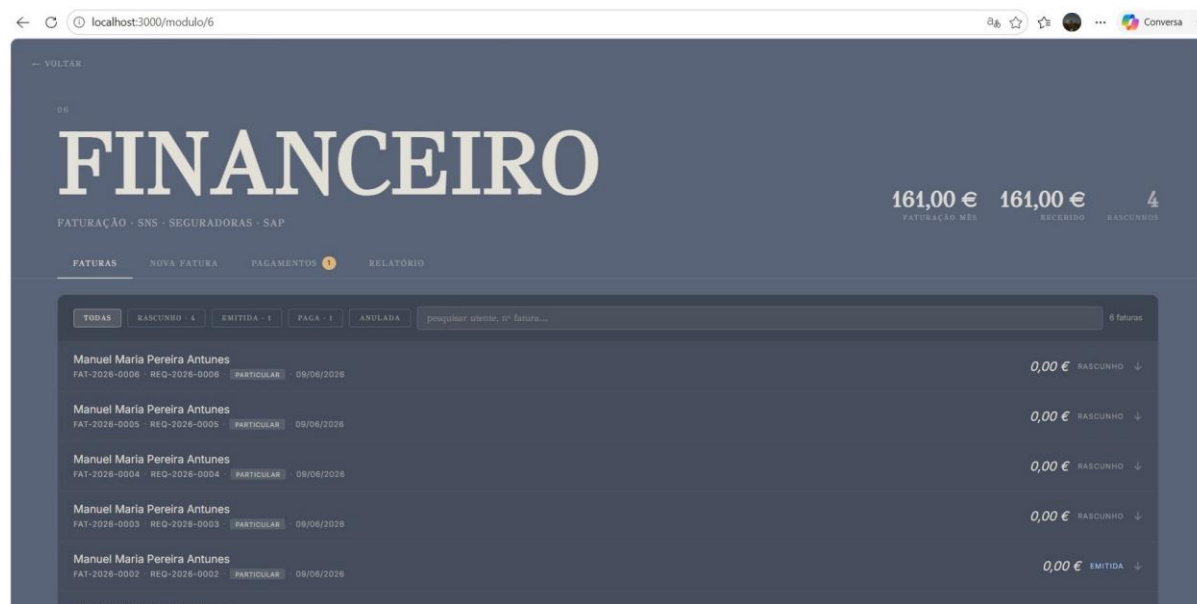


Figura 23 – Módulo financeiro.

8.14. Módulo Equipamentos

O módulo de equipamentos (Figura 24) gere equipamentos, reagentes e manutenções, com

alertas de stock e de calibração.

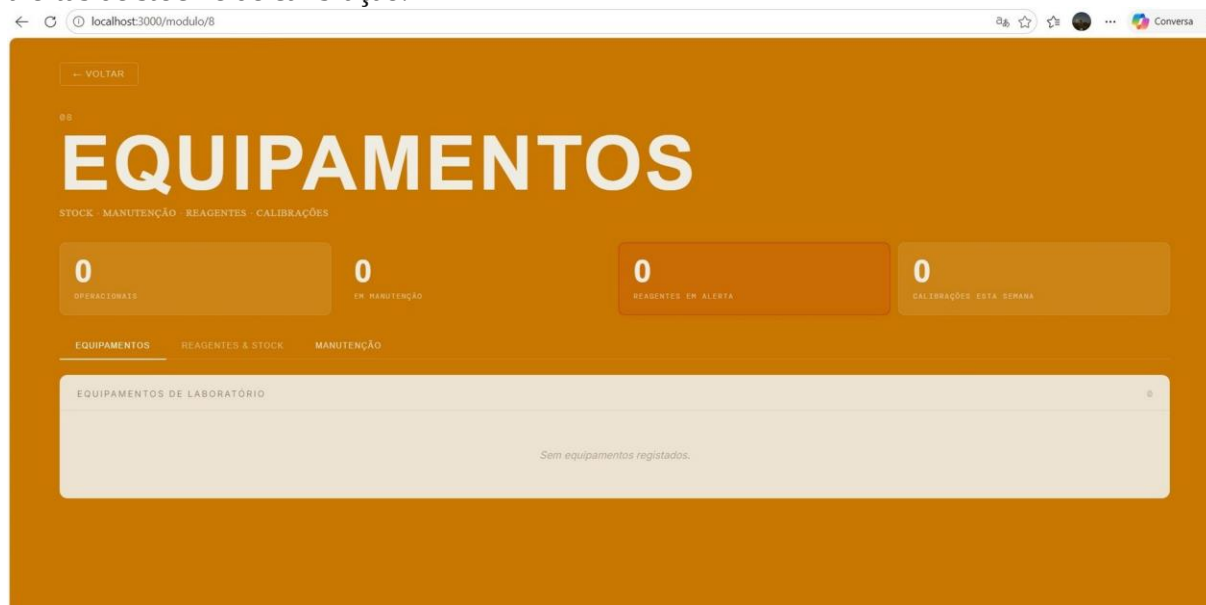


Figura 24 – Módulo de equipamentos e reagentes.

8.15. Módulo Analytics

O painel de analytics (Figura 25) apresenta indicadores e gráficos operacionais, construídos com a biblioteca Recharts a partir de agregações no MongoDB.

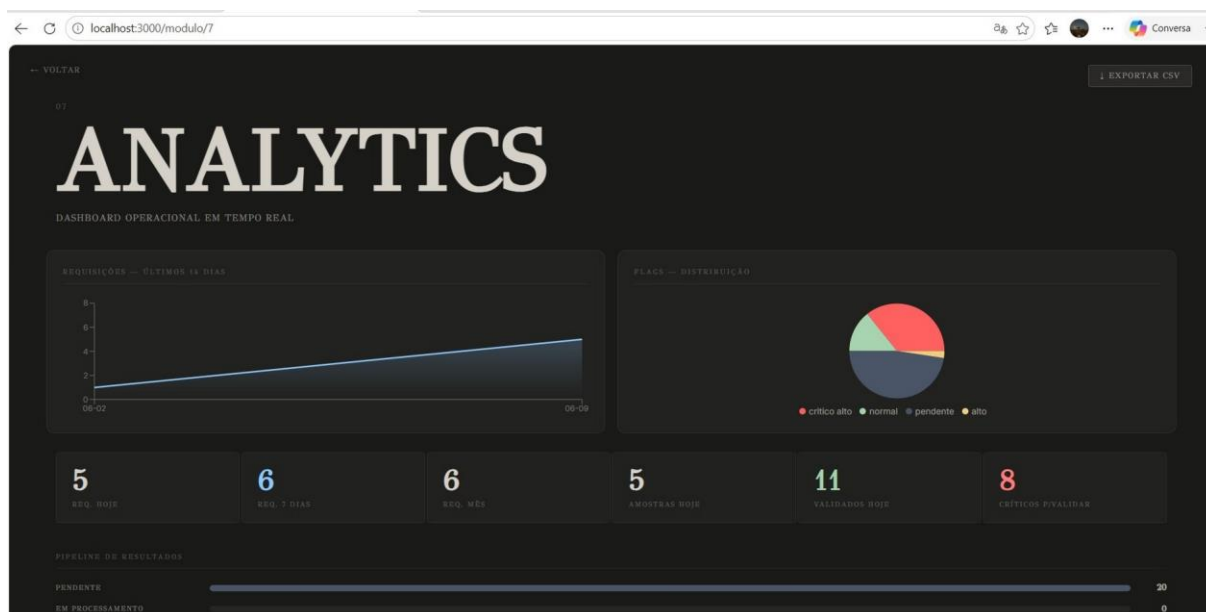


Figura 25 – Módulo de analytics.

8.16. Módulo Utilizadores

O módulo de utilizadores (Figura 26) permite a gestão de contas, a edição de papéis e a aprovação de registos pendentes.

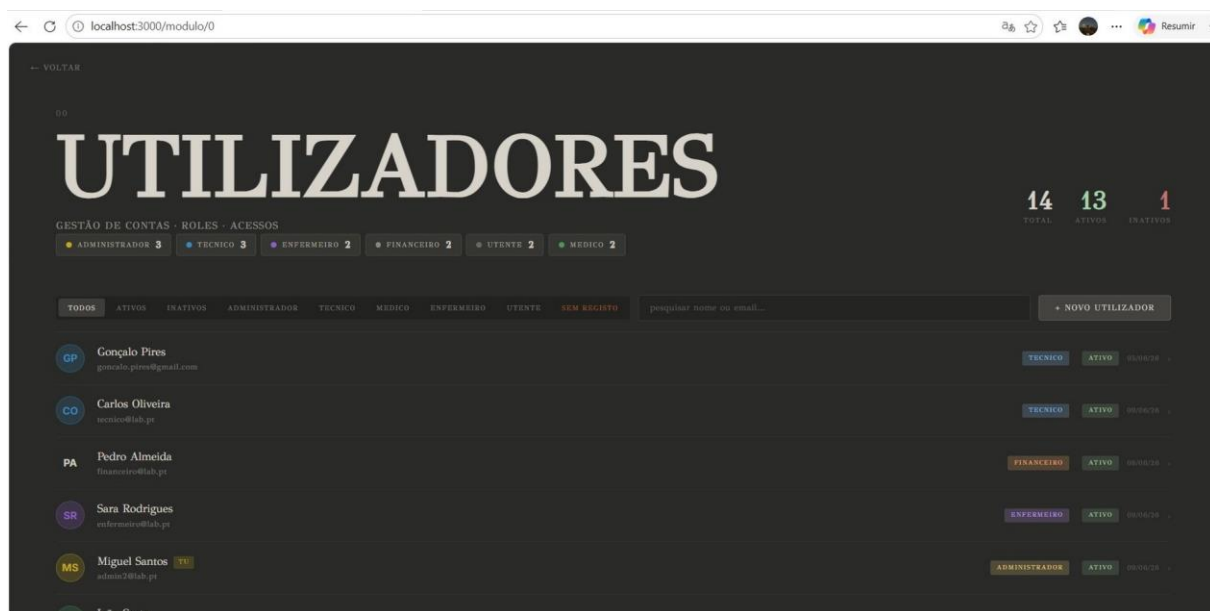


Figura 26 – Módulo de gestão de utilizadores.

8.17. Módulo Segurança

O módulo de segurança (Figura 27) disponibiliza o histórico de ações, com o registo de eventos por utilizador e por módulo.

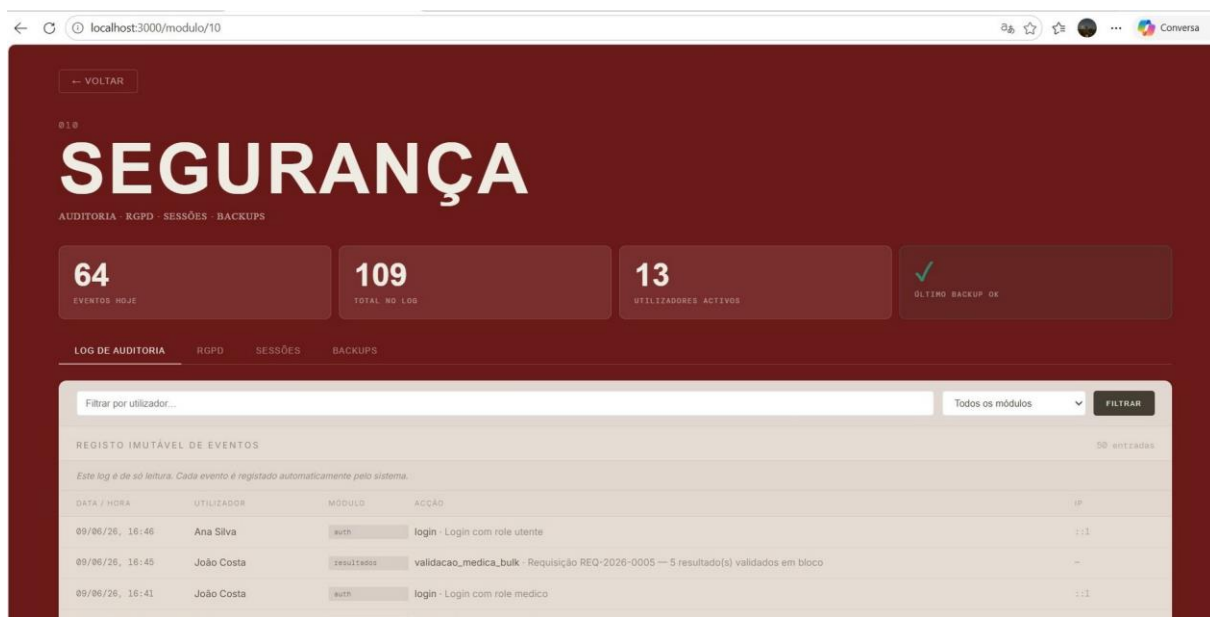


Figura 27 – Módulo de segurança — histórico de ações.

8.18. Módulo Integrações

O módulo de integrações (Figura 28) expõe o estado das ligações externas, incluindo a prescrição eletrónica nacional, a API REST documentada e os webhooks para sistemas externos.



Figura 28 – Módulo de integrações.

8.19. Notificações em Tempo Real

O sistema integra um mecanismo de notificações construído sobre a própria API REST. O sino presente no cabeçalho consulta periodicamente o servidor, por sondagem HTTP (polling), apresentando um contador de mensagens não lidas. Cada notificação é persistida na base de dados e dirigida ao respetivo destinatário (novos resultados, valores críticos, entre outros), ficando acessível na lista de notificações (Figura 29).

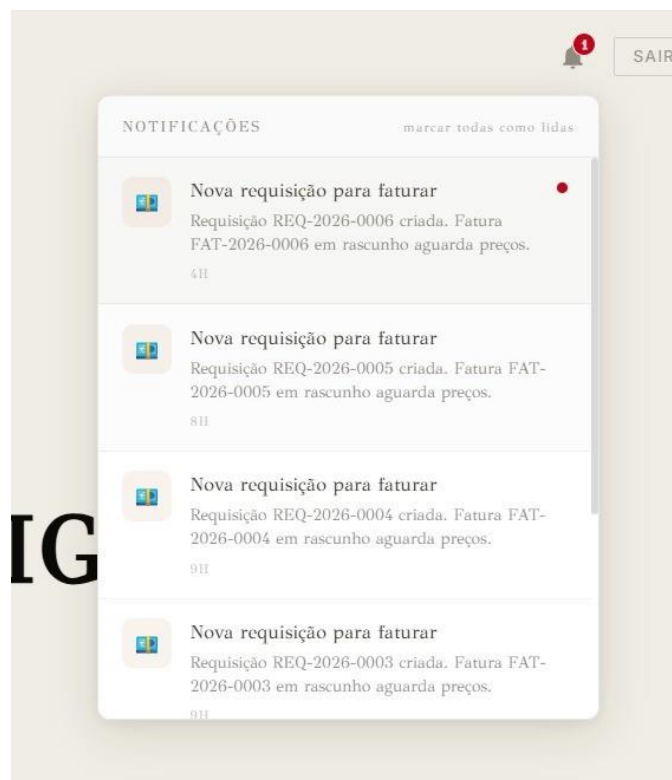


Figura 29 – Painel de notificações, com contador de não lidas.

8.20. Intercâmbio de dados em XML

A funcionalidade de intercâmbio em XML está disponível nas interfaces dos perfis com permissão para a utilizar. No portal financeiro, gerido pelo administrador, existem ações para exportar e importar as faturas em XML; no portal técnico, uma barra de intercâmbio permite exportar e importar requisições e resultados. Em ambos os casos, ao importar um ficheiro inválido é apresentado um painel com a lista de erros de validação (mensagem e linha), sem qualquer alteração aos dados. A Figura 30 representa o fluxo de importação, evidenciando o passo de validação contra o esquema XSD antes da gravação e o ramo alternativo de erro, em que o servidor responde com o código 422 e a lista de erros, mantendo a base de dados intacta.

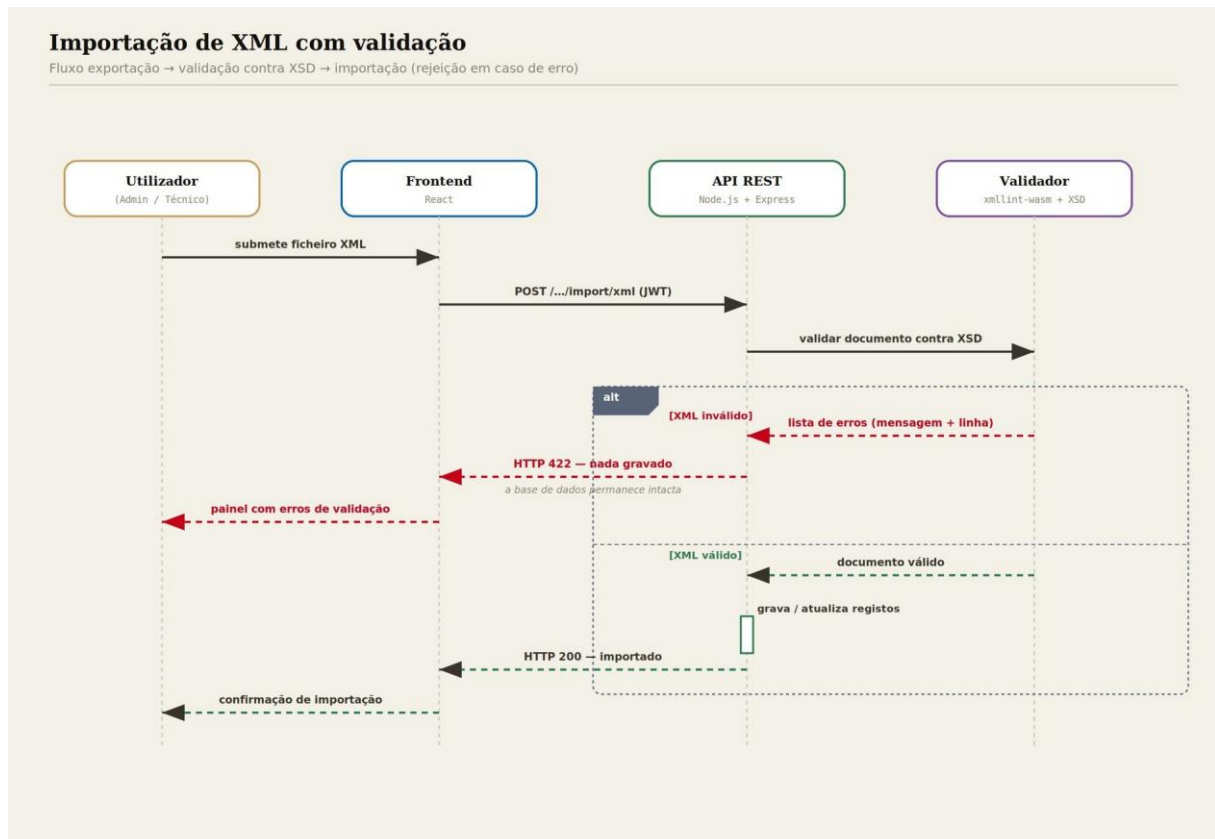


Figura 30 – Importação de XML com validação contra o esquema XSD (com ramo de erro 422).

9. Testes

9.1. Estratégia e ferramenta de teste

A verificação foi efetuada através de validação manual da API com o cliente REST Postman. Foram preparadas coleções de pedidos que cobrem a autenticação e os principais recursos do sistema, verificando-se, em cada caso, o código de estado HTTP devolvido, o corpo da resposta e a correta aplicação das regras de autorização por papel.

9.2. Casos de teste implementados

Foram implementados testes de integração para três domínios: autenticação, requisições e resultados, conforme a Tabela 11.

Tabela 11 - Casos de teste de integração.

Grupo	Casos de teste
Autenticação	Rejeição de credenciais inválidas; rejeição de corpo vazio; aceitação de credenciais válidas; verificação da estrutura e dos campos do token JWT.
Requisições	Rejeição de pedidos sem token; acesso autorizado com token válido; verificação da paginação; criação de requisição pelo próprio utente.
Resultados	Rejeição sem token; acesso do técnico; contadores de estatísticas; aplicação do RBAC na
	validação (um utente ou médico não pode validar tecnicamente — resposta 403).

A título de exemplo, o teste de RBAC sobre os resultados verifica explicitamente que um pedido de validação técnica efetuado por um perfil não autorizado é recusado com o código de estado 403 (Forbidden) antes de alcançar o recurso, confirmando a correta aplicação da matriz de permissões.

10. Conclusão

10.1. Síntese e objetivos atingidos

O LabSystem cumpre os objetivos propostos: implementa, de ponta a ponta, o fluxo clínico-laboratorial, com controlo de acessos por papel, dupla validação de resultados, notificação de valores críticos, faturação com comparticipação e um trilha de auditoria orientado ao RGPD. A arquitetura cliente-servidor desacoplada, assente numa API REST e numa base de dados na cloud (MongoDB Atlas), resulta num sistema coerente, seguro e de manutenção simples.

10.2. Limitações e trabalho futuro

Identificam-se como limitações o carácter ainda parcial de algumas integrações externas e a cobertura de testes, concentrada nos domínios críticos. Como trabalho futuro, propõe-se: a conclusão do módulo de controlo de qualidade (calibrações e conformidade ISO), cuja implementação ficou em estudo; a implementação completa das integrações de prescrição eletrónica e de exportação contabilística (SAP); o alargamento da cobertura de testes; e a consolidação das funcionalidades de notificação automática (e-mail e SMS). Estas extensões aproximariam o LabSystem de uma solução pronta para ambiente de produção.

Referências

- [1] E. Coiera, Guide to Health Informatics, 3.^a ed. Boca Raton: CRC Press, 2015.
- [2] J. H. van Bommel e M. A. Musen, Handbook of Medical Informatics. Heidelberg: Springer, 1997.
- [3] Microsoft, “TypeScript Documentation,” [Online]. Disponível em: <https://www.typescriptlang.org/docs>.
- [4] OpenJS Foundation, “Node.js — JavaScript runtime,” [Online]. Disponível em: <https://nodejs.org>.
- [5] OpenJS Foundation, “Express — Web framework for Node.js,” [Online]. Disponível em: <https://expressjs.com>.
- [6] MongoDB Inc., “MongoDB Documentation,” [Online]. Disponível em: <https://www.mongodb.com/docs>.

- [7] Automattic, “Mongoose ODM Documentation,” [Online]. Disponível em: <https://mongoosejs.com>.
- [8] Meta Open Source, “React — A JavaScript library for building user interfaces,” [Online]. Disponível em: <https://react.dev>.
- [9] M. Jones, J. Bradley e N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
- [10] Regulamento (UE) 2016/679 (RGPD), Jornal Oficial da União Europeia, 2016.

Anexo A - Esquemas XSD

Apresentam-se os três esquemas XSD que definem e validam a estrutura dos documentos XML trocados pelo sistema.

Anexo A.1 - fatura.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="faturas" type="FaturasType"/>
  <xs:complexType name="FaturasType">
    <xs:sequence>
      <xs:element name="fatura" type="FaturaType" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="periodo" type="xs:string" use="optional"/>
  </xs:complexType>
  <xs:complexType name="FaturaType">
    <xs:sequence>
      <xs:element name="numeroFatura" type="xs:string"/>
      <xs:element name="dataEmissao" type="xs:date"/>
      <xs:element name="utenteNome" type="xs:string"/>
      <xs:element name="tipo" type="TipoFaturaType"/>
      <xs:element name="seguradora" type="xs:string" minOccurs="0"/>
      <xs:element name="valorBruto" type="xs:decimal"/>
      <xs:element name="percentComp participacao" type="xs:decimal" minOccurs="0"/>
      <xs:element name="valorLiquido" type="xs:decimal"/>
      <xs:element name="estado" type="EstadoFaturaType"/>
    </xs:sequence>
  </xs:complexType>
  <xs:simpleType name="TipoFaturaType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="particular"/>
      <xs:enumeration value="sns"/>
      <xs:enumeration value="seguradora"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType name="EstadoFaturaType">
    <xs:restriction base="xs:string">
      <xs:enumeration value="rascunho"/>
      <xs:enumeration value="emitida"/>
      <xs:enumeration value="paga"/>
      <xs:enumeration value="anulada"/>
    </xs:restriction>
  </xs:simpleType>
</xs:schema>
```

Anexo A.2 - requisicao.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xs:element name="requisicao" type="RequisicaoType"/>
  <xs:element name="requisicoes">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="requisicao" type="RequisicaoType" minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
      <xs:attribute name="exportadoEm" type="xs:dateTime" use="optional"/>
    </xs:complexType>
  </xs:element>
  <xs:complexType name="RequisicaoType">
    <xs:sequence>
      <xs:element name="numeroRequisicao" type="xs:string"/>
      <xs:element name="dataEmissao" type="xs:dateTime"/>
      <xs:element name="urgente" type="xs:boolean" default="false"/>
      <xs:element name="estado" type="EstadoReqType"/>
      <xs:element name="utente" type="UtenteRefType"/>
      <xs:element name="medico" type="xs:string" minOccurs="0"/>
      <xs:element name="analises" type="AnalisesType"/>
      <xs:element name="observacoes" type="xs:string" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:string" use="optional"/>
  </xs:complexType>
  <xs:complexType name="UtenteRefType">
    <xs:sequence>
      <xs:element name="numeroProcesso" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
```

```

        <xs:element name="nome" type="xs:string"/>
        <xs:element name="sns" type="xs:string" minOccurs="0"/>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="AnalisesType">
    <xs:sequence>
        <xs:element name="analise" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:element name="codigo" type="xs:string"/>
                    <xs:element name="nome" type="xs:string"/>
                </xs:sequence>
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:simpleType name="EstadoReqType">
    <xs:restriction base="xs:string">
        <xs:enumeration value="pendente"/>
        <xs:enumeration value="em_curso"/>
        <xs:enumeration value="concluida"/>
        <xs:enumeration value="cancelada"/>
    </xs:restriction>
</xs:simpleType>
</xs:schema>

```

Anexo A.3 - resultado.xsd

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
    <xs:element name="resultado" type="ResultadoType"/>
    <xs:element name="resultados">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="resultado" type="ResultadoType" minOccurs="0" maxOccurs="unbounded"/>
            </xs:sequence>
            <xs:attribute name="exportadoEm" type="xs:dateTime" use="optional"/>
        </xs:complexType>
    </xs:element>
    <xs:complexType name="ResultadoType">
        <xs:sequence>
            <xs:element name="codigoResultado" type="xs:string"/>
            <xs:element name="requisicaoNumero" type="xs:string"/>
            <xs:element name="amostraCodigo" type="xs:string"/>
            <xs:element name="utenteNome" type="xs:string"/>
            <xs:element name="analiseCodigo" type="xs:string"/>
            <xs:element name="analiseNome" type="xs:string"/>
            <xs:element name="valor" type="xs:string" minOccurs="0"/>
            <xs:element name="unidade" type="xs:string" minOccurs="0"/>
            <xs:element name="flag" type="FlagResType"/>
            <xs:element name="estado" type="EstadoResType"/>
            <xs:element name="dataValidacao" type="xs:dateTime" minOccurs="0"/>
            <xs:element name="observacoes" type="xs:string" minOccurs="0"/>
        </xs:sequence>
        <xs:attribute name="id" type="xs:string" use="optional"/>
    </xs:complexType>
    <xs:simpleType name="FlagResType">
        <xs:restriction base="xs:string">
            <xs:enumeration value="pendente"/>
            <xs:enumeration value="normal"/>
            <xs:enumeration value="alto"/>
            <xs:enumeration value="baixo"/>
            <xs:enumeration value="critico_alto"/>
            <xs:enumeration value="critico_baixo"/>
        </xs:restriction>
    </xs:simpleType>
    <xs:simpleType name="EstadoResType">
        <xs:restriction base="xs:string">
            <xs:enumeration value="pendente"/>
            <xs:enumeration value="em_processamento"/>
            <xs:enumeration value="resultado_disponivel"/>
            <xs:enumeration value="validado_tecnico"/>
            <xs:enumeration value="validado_medico"/>
            <xs:enumeration value="rejeitado"/>
        </xs:restriction>
    </xs:simpleType>

```

```
</xs:schema>
```

Anexo B - Exemplos de XML

Exemplos de instâncias XML geradas pela exportação e validadas com sucesso contra os esquemas do Anexo A, seguidos de um exemplo inválido e da respetiva rejeição pela API.

Anexo B.1 - Exportação de faturas

```
<?xml version="1.0" encoding="UTF-8"?>
<faturas periodo="2026-06">
  <fatura>
    <numeroFatura>FAT-2026-0001</numeroFatura>
    <dataEmissao>2026-06-10</dataEmissao>
    <utenteNome>Ana & Silva</utenteNome>
    <tipo>particular</tipo>
    <valorBruto>45.00</valorBruto>
    <valorLiquido>45.00</valorLiquido>
    <estado>emitida</estado>
  </fatura>
  <fatura>
    <numeroFatura>FAT-2026-0002</numeroFatura>
    <dataEmissao>2026-06-11</dataEmissao>
    <utenteNome>Bruno Costa</utenteNome>
    <tipo>seguradora</tipo>
    <seguradora>Médias</seguradora>
    <valorBruto>100.00</valorBruto>
    <percentComparticipacao>40.00</percentComparticipacao>
    <valorLiquido>60.00</valorLiquido>
    <estado>rascunho</estado>
  </fatura>
</faturas>
```

Anexo B.2 - Exportação de requisições

```
<?xml version="1.0" encoding="UTF-8"?>
<requisicoes exportadoEm="2026-06-14T09:23:34.111Z">
  <requisicao>
    <numeroRequisicao>REQ-2026-0001</numeroRequisicao>
    <dataEmissao>2026-06-10T00:00:00.000Z</dataEmissao>
    <urgente>true</urgente>
    <estado>pendente</estado>
    <utente>
      <numeroProcesso>P123</numeroProcesso>
      <nome>Ana & Silva</nome>
    </utente>
    <medico>Dr. Costa</medico>
    <analises>
      <analise><codigo>BI001</codigo><nome>Glicose</nome></analise>
      <analise><codigo>HEM01</codigo><nome>Leucócitos</nome></analise>
    </analises>
    <observacoes>jejum</observacoes>
  </requisicao>
</requisicoes>
```

Anexo B.3 - Exportação de resultados

```
<?xml version="1.0" encoding="UTF-8"?>
<resultados exportadoEm="2026-06-14T09:23:34.308Z">
  <resultado>
    <codigoResultado>RES-0001</codigoResultado>
    <requisicaoNumero>REQ-2026-0001</requisicaoNumero>
    <amostraCodigo>AM-1</amostraCodigo>
    <utenteNome>Ana & Silva</utenteNome>
    <analiseCodigo>BI001</analiseCodigo>
    <analiseNome>Glicose</analiseNome>
    <valor>92</valor>
    <unidade>mg/dL</unidade>
    <flag>normal</flag>
    <estado>validado_tecnico</estado>
    <dataValidacao>2026-06-12T00:00:00.000Z</dataValidacao>
  </resultado>
</resultados>
```

Anexo B.4 - Exemplo de XML inválido e respetiva rejeição

Documento de faturas com um valor de tipo fora da enumeração permitida (cartao não

pertence ao conjunto particular | sns | seguradora). Os números à esquerda são apenas a referência de linha.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <faturas periodo="2026-06">
3   <fatura>
4     <numeroFatura>FAT-2026-0003</numeroFatura>
5     <dataEmissao>2026-06-12</dataEmissao>
6     <utenteNome>Carla Nunes</utenteNome>
7     <tipo>cartao</tipo>          <!-- valor inválido -->
8     <valorBruto>30.00</valorBruto>
9     <valorLiquido>30.00</valorLiquido>
10    <estado>emitida</estado>
11  </fatura>
12 </faturas>
```

Mensagem de validação produzida pelo XSD (linha 7):

Element 'tipo': [facet 'enumeration'] The value 'cartao' is not an element of the set {'particular', 'sns', 'seguradora'}.

Resposta HTTP devolvida pela API ao importar (estado 422 Unprocessable Entity):

```
{
  "message": "XML inválido - não foi importado nada.",
  "errors": [
    {
      "message": "Element 'tipo': [facet 'enumeration'] The value 'cartao' is not an element of the
set {'particular', 'sns', 'seguradora'}.",
      "line": 7
    }
  ]
}
```